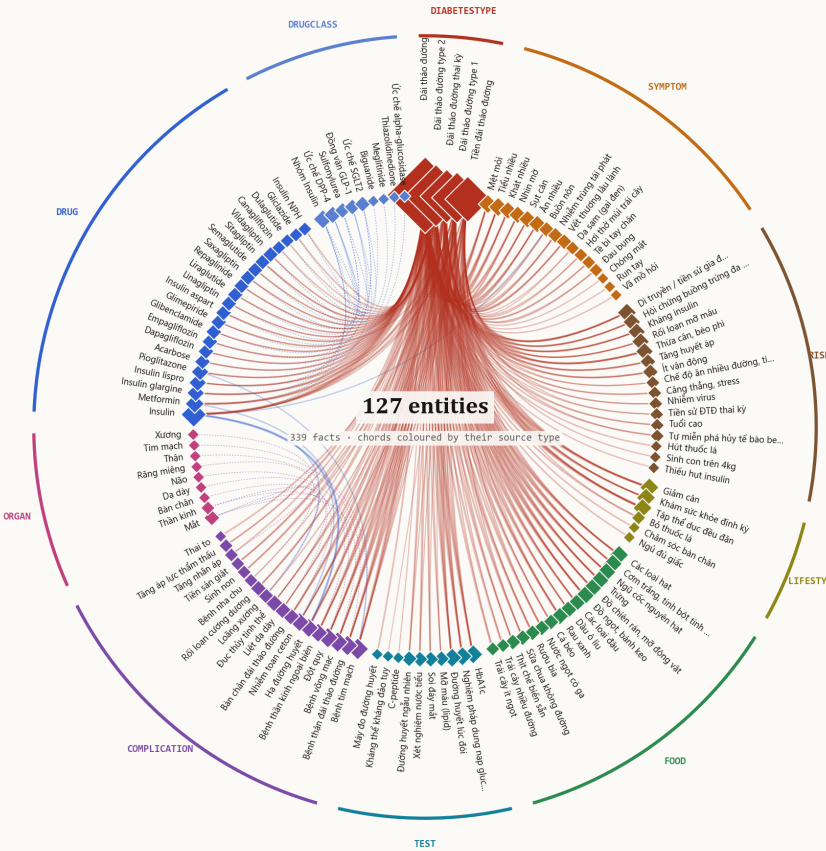
GRAPH 01 ♦ INTELLIGENT SYSTEM DEVELOPMENT

POSTS AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY · FACULTY OF INFORMATION TECHNOLOGY

Glyco *Atlas*

A knowledge graph of diabetes, charted from 1,096 Vinmec health articles and explored through Neo4j.



STUDENT	**Bùi Nguyên Hoàng Việt**
ID	**B23DCDT285**
LECTURER	Assoc. Prof. Dinh Que Tran
EMAIL	quetd@ptit.edu.vn
SEMESTER	I.2025 -- 2026
TYPE	Individual project
PLACE	Hanoi, 2026

Contents

At a Glance . . . 3

1 Why Chart Diabetes Knowledge? . . . 4
- 1.1 Knowledge locked in prose . . . 4
- 1.2 Where keyword search runs out . . . 4
- 1.3 Facts as arrows . . . 4
- 1.4 What this project delivers . . . 5
- 1.5 The system in one picture . . . 5

2 The Brief . . . 6
- 2.1 Input and output . . . 6
- 2.2 Questions the atlas must answer . . . 6
- 2.3 How success is judged . . . 7
- 2.4 What it is not . . . 7

3 Gathering the Articles . . . 8
- 3.1 Why Vinmec . . . 8
- 3.2 Finding the pages through the sitemap . . . 8
- 3.3 The accent trap, and a second filter . . . 8
- 3.4 Parsing a page: keep the headings . . . 9
- 3.5 The corpus at a glance . . . 10

4 From Prose to Triples . . . 11
- 4.1 Triples, nodes and edges . . . 11
- 4.2 Schema: entity types and relation types . . . 12
- 4.3 Extraction method . . . 13
  - 4.3.1 Why a bullet is not a node . . . 13
  - 4.3.2 Vocabulary and entity normalisation . . . 14
  - 4.3.3 Finding an alias in the text . . . 14
  - 4.3.4 The section rule: the heading decides . . . 14
  - 4.3.5 Which diabetes is the subject? . . . 16
  - 4.3.6 Counting evidence: one page is not enough . . . 16
  - 4.3.7 Foods: recommended or to avoid . . . 17
  - 4.3.8 Drug side effects . . . 17
  - 4.3.9 Background facts added by hand . . . 17
- 4.4 Output files . . . 17
- 4.5 Scorecard against the lecture . . . 18

5 Under the Hood . . . 18
- 5.1 Project layout . . . 18
- 5.2 Neo4j . . . 19
  - 5.2.1 Why a graph database . . . 19
  - 5.2.2 Data model . . . 19
  - 5.2.3 Loading . . . 19
- 5.3 Cypher . . . 20
- 5.4 Bounded retrieval . . . 20
- 5.5 The website . . . 21
  - 5.5.1 Architecture . . . 21
  - 5.5.2 API . . . 21
  - 5.5.3 Web client . . . 22

6 Seeing the Graph . . . 23
- 6.1 Size of the graph . . . 23
- 6.2 The whole atlas on a ring . . . 24
- 6.3 Close-ups . . . 25
- 6.4 Type by type . . . 27
- 6.5 What matters most . . . 28

7 Putting the Graph to Work . . . 30
7.1 One hop: everything about Metformin . . . 30
7.2 Two hops: which organs can type 2 damage? . . . 30
7.3 Type 1 against type 2 . . . 31
7.4 A symptom finder . . . 31
7.5 Diagnosis: tests and thresholds . . . 32
7.6 Diet in gestational diabetes . . . 33
7.7 Provenance: where does a fact come from? . . . 33
7.8 Written about together . . . 34
8 How Far to Trust It . . . 35
8.1 What was measured, and how . . . 35
8.2 Result . . . 35
8.3 How much evidence the facts carry . . . 36
9 A Tour of Glyco Atlas . . . 37
9.1 Finding your way in . . . 38
9.2 Everything about one entity, with its sources . . . 40
9.3 Comparing the types . . . 41
9.4 Checking symptoms . . . 42
9.5 Tracing a link . . . 43
9.6 On a phone . . . 44
10 Where It Goes Wrong . . . 45
10.1 The two errors . . . 45
10.2 The fix . . . 45
10.3 What the fix cost: precision against recall . . . 46
10.4 Other known errors . . . 46
11 Known Blind Spots . . . 47
12 Takeaways and Next Steps . . . 48
12.1 What was achieved . . . 48
12.2 Next steps . . . 48
A Rebuilding Everything . . . 49

At a Glance

127	339	1,096	0.96
ENTITIES OF 10 TYPES	FACTS, 12 RELATIONS	SOURCE ARTICLES	PRECISION ON A 50-FACT SAMPLE

Diabetes is written about everywhere and queried nowhere. The hospital group **Vinmec** publishes about a thousand Vietnamese articles on it, but the knowledge sits inside paragraphs. This project turns those paragraphs into a **knowledge graph**: a network of small, explicit facts such as *(Đái tháo đường type 2, HAS_SYMPTOM, Khát nhiều)* or *(Metformin, BELONGS_TO_CLASS, Biguanide)*. The facts live in the graph database **Neo4j**, can be asked questions in **Cypher**, and can be browsed by anyone in a small website, **Glyco Atlas**, that shows every fact beside the articles that state it.

The pipeline in four moves. ♦

1. **Crawl.** Read Vinmec's sitemap, select 1,197 candidate pages whose address mentions diabetes, download them politely and keep the 1,096 that really discuss the disease (about 1.13 million words).
2. **Extract.** Read each article section by section. The *heading* (*Triệu chứng*, *Nguyên nhân*, *Biến chứng*, *Điều trị*…) decides which kind of fact the text below it may hold; a hand-written vocabulary of 106 concepts in 320 spellings supplies the things.
3. **Vote.** Each article votes for the facts it states. A fact is kept only when at least two different articles agree.
4. **Serve.** Load the result into Neo4j (127 entities, 339 facts, 12 relation types, linked to the 1,096 articles) and expose it through Cypher, a bounded retrieval function and the website.

What the project found. ♦

- **Headings are the key signal.** Health articles follow standard headings, and using them turns free Vietnamese text into typed facts with no language model. A word in the wrong section ("mệt mỏi" in a breakfast article) is simply ignored.
- **Agreement removes noise.** Requiring two articles drops one-off mentions, and the stored support count lets the reader choose how cautious to be: 252 extracted facts have three or more articles behind them, 70 have ten or more.
- **The facts hold up.** In a random sample of 50 extracted facts, each checked against all its sources, **48 were correct** (0.96; likely range 0.87–0.99). Both errors had one cause, which led to a stricter heading rule and a clear look at what that rule costs.
- **The graph answers what no article answers.** Two arrows (type 2 *can cause* diabetic kidney disease, which *affects* the kidney) answer "which organs can type 2 damage?", and every step traces back to its articles.

The work covers every requirement of the lecture's Mini Project 3 (at least 50 entities, 100 relationships, 5 relationship types, 3 entity types, 5 Cypher queries) and of Exercise 1 (triples as CSV, a visualisation, one-hop and two-hop queries).

PART 1

Why Chart Diabetes Knowledge?

1.1 Knowledge locked in prose

Diabetes is among the most common chronic diseases in Vietnam, and most people learn about it by reading articles online. Vinmec, one of the country's largest private hospital groups, has published close to a thousand Vietnamese articles on diabetes alone: symptoms, risk groups, complications, diagnosis, drugs, diet.

That knowledge is valuable and hard to reach:

- A reader who wants to know *which complications of type 2 diabetes affect the eyes* must find the right articles, read them and assemble the answer personally.
- The same symptom is written many ways (*uống nhiều*, *khát nước*, *hay khát*).
- An answer is often split across articles: one says type 2 can cause retinopathy, another that retinopathy damages the retina.
- For a program, an article is only a long sequence of characters.

The goal is to put the collection into a form a program can query directly and a person can explore without reading hundreds of pages, **while keeping the link to the original articles** so every answer can be checked.

1.2 Where keyword search runs out

Keyword search is the obvious tool, and here it fails three ways.

One idea, many spellings. ♦Searching *khát nước* (thirsty) misses articles that write *uống nhiều* or *khát nhiều*; searching *Metformin* misses the brand name *Glucophage*. The reader would have to know every synonym first.

A word is not a fact. ♦A breakfast article may say a good meal "helps you feel less *mệt mỏi*" (tired). Searching *mệt mỏi* finds it, yet the article never says fatigue is a symptom of diabetes. Search finds where a word appears, not what the text claims about it.

Answers that need two facts. ♦No article says "type 2 diabetes damages the kidney" in that form. The answer comes from joining "type 2 can cause diabetic kidney disease" with "diabetic kidney disease affects the kidney". Search returns documents one at a time and cannot join statements across them.

1.3 Facts as arrows

A knowledge graph stores knowledge as many small **facts**, each connecting exactly two things through a named relationship: *(subject, relation, object)*. Draw the things as dots (**entities**, or nodes) and the facts as arrows (**relationships**, or edges) and the whole collection becomes a network. That representation answers the three problems directly:

- **Synonyms are settled once.** *uống nhiều*, *khát nước* and *hay khát* all become one entity, *Khát nhiều*, at the moment the facts are made.
- **Facts are explicit and typed.** A fact exists only if an article states it where such statements are made (Part 4.3).
- **Facts can be joined by walking.** A question becomes a walk: start at *type 2*, follow *can cause* to the complications, then *affects* to the organs. The path itself is the explanation.

The lecture phrases the contrast well: keyword or vector retrieval asks "which passages resemble the question?", graph retrieval asks "which entities are explicitly connected?". For questions about how

symptoms, causes, complications, tests and treatments relate, the second question is the right one.

WHY IT MATTERS IN MEDICINE

> A graph also keeps something a trained model loses: **provenance**. Every extracted fact here stores the articles behind it, so any answer can be checked against its sources. When a wrong answer can mislead a patient, that is not a nicety.

1.4 What this project delivers

Table 1 lists the requirements, taken from the task and from the lecture's Mini Project 3 and Exercise 1, with the part of this report that addresses each.

Table 1. Requirements and where they are met.

#	REQUIREMENT	PART
R1	Collect diabetes data from Vinmec	3
R2	Build a knowledge graph: $\geq$ 50 entities, $\geq$ 100 relationships, $\geq$ 5 relationship types, $\geq$ 3 entity types	4, 4.5
R3	Store triples as CSV (`subject, relation, object`)	4.4
R4	Store and query the graph in Neo4j; at least 5 Cypher queries incl. one-hop and two-hop	5, 7
R5	Visualise the graph	6
R6	Graph retrieval from an entity, within 1–2 hops	5.4
R7	A website for retrieving the data	5.5, 9
R8	Evaluate the system systematically	8
R9	Analyse errors and limitations	10, 11

The report follows the lecture's *Recommended Final Report Structure* (§37 of *From Deep Learning to Knowledge Graph, RAG and Intelligent Agents*). That outline also covers a deep-learning model, a RAG design and an agent design; this project is the knowledge-graph-plus-retrieval part, so those three are left out. Two parts are added because they carry a lot of the work: **Seeing the Graph** (Part 6) and **A Tour of Glyco Atlas** (Part 9).

1.5 The system in one picture

Figure 1 shows the five programs. Each reads what the one before it wrote to disk, and the stages are kept apart on purpose:

- **Crawling** takes about twenty minutes and needs the site to be reachable, so it runs once.
- **Extraction** is where the rules were revised and re-tested several times, so it has to be cheap to re-run with no downloading.
- **Loading** can rebuild the database from the CSV files whenever needed.
- The **website** only reads from the database.

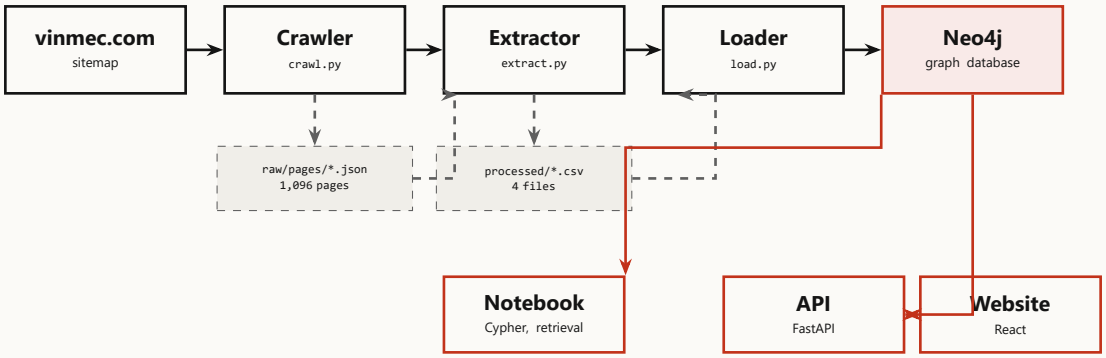


Figure 1. The system. Solid arrows: data moving between programs. Dashed arrows: files written to and read from disk. The notebook, the Cypher queries and the website all read the same Neo4j database.

PART 2

The Brief

2.1 Input and output

Input. ♦Public Vietnamese articles of vinmec.com about diabetes: the official disease pages (short structured summaries with fixed headings for overview, causes, symptoms, risk groups, prevention, diagnosis and treatment) and the far larger set of news and advice articles written or reviewed by Vinmec doctors. No other source.

Output. ♦A knowledge graph in which

- every node is a named diabetes concept with a type: diabetes type, symptom, risk factor, complication, test, drug, drug class, food, lifestyle measure or organ;
- every edge is a fact with a named relation such as `HAS_SYMPTOM` or `TREATED_BY`;
- every extracted fact records its **support**, the number of different articles that state it, and which articles they are;

plus three ways to use it: Cypher queries in Neo4j, a retrieval function that returns the facts around one entity, and a website.

2.2 Questions the atlas must answer

A graph is only as useful as the questions it can take. The design started from what a reader of diabetes articles would ask, and each question needs a different kind of query (Table 2).

Table 2. Questions, and the kind of graph query each one needs.

QUESTION	QUERY TYPE	WHAT THE GRAPH MUST HOLD
Symptoms of type 1? Drugs for type 2?	one-hop	typed facts per diabetes type
What do we know about Metformin?	one-hop, both directions	facts where Metformin is subject *or* object
Which organs can type 2 damage?	two-hop	complications *and* the organ each affects
Which types match these symptoms?	aggregation over one hop	symptom facts per type, with support
How does obesity connect to the kidney?	shortest path	a connected graph
How is diabetes diagnosed, at what values?	one-hop + node property	tests, with thresholds on the nodes
Where does this fact come from?	provenance	article URLs on each fact

These questions drove the design. The two-hop one is why the hand-added *affects organ* facts exist (Part 4.3.9); the symptom finder is why support is stored on every fact instead of being used once as a filter; provenance is why the articles are nodes in the graph.

2.3 How success is judged

1. **Size and structure.** The lecture minimums, triples in CSV, at least five Cypher queries including one-hop and two-hop, a visualisation.
2. **Correctness, measured.** A random sample of facts is checked by hand against the sources, and precision is reported with its uncertainty.
3. **Traceability.** Every extracted fact links to the articles that state it.
4. **Usability.** Someone with no Cypher can find an entity, see its facts and sources, compare types, check symptoms and trace connections, all in a browser.

2.4 What it is not

The atlas summarises what public health articles say. It is not a diagnostic tool and not medical advice. "Correct" in this report means *the Vinmec articles state it for that type of diabetes*, not that a doctor verified it against current guidelines. The website and the evaluation repeat this on purpose.

PART 3

Gathering the Articles

3.1 Why Vinmec

Three properties make Vinmec a good single source for a Vietnamese diabetes graph:

- **Language.** The articles are in Vietnamese, so the graph uses the words Vietnamese readers use.
- **Coverage.** Almost a thousand diabetes articles, from first signs to diet, drugs and pregnancy.
- **Consistency.** They are written or reviewed by the hospital's doctors and follow a recognisable layout, with headings such as *Triệu chứng* (symptoms), *Nguyên nhân* (causes) and *Điều trị* (treatment). Part 4.3 shows that this layout is what makes automatic extraction possible.

3.2 Finding the pages through the sitemap

Not the search box. ♦The site search is built by JavaScript *after* the page loads, so a program that downloads the search page receives the page before any results exist. A test request for "tiểu đường" returned a 150 KB page with no article links.

The sitemap instead. ♦Large sites publish a **sitemap**, a list of every URL, for search engines. Vinmec's `sitemap.xml` points to about 230 smaller ones. The crawler reads the Vietnamese article maps (`posts-vi-1.xml` … `posts-vi-87.xml`) and the disease-page map (`diseases-vi-1.xml`): about 222,000 URLs, the whole Vietnamese site.

Selecting by slug. ♦A URL ends with a readable name for the page, its **slug**, written without accents: the article *"Bệnh đái tháo đường: chế độ ăn như thế nào?"* has the slug `benh-dai-thao-duong-che-do-nhu-nao-vi`. Matching keywords in slugs picks diabetes pages without downloading all 222,000. The keywords come in two strengths, because they carry different risks:

- **strong** (`tieu-duong`, `thao-duong`, `dai-duong`): almost certainly diabetes;
- **weak** (`insulin`, `metformin`, `hba1c`, `duong-huyet` = blood sugar, `glucose`, `sglt2`…): the page *might* be about diabetes, but these words also turn up in articles on the hormone's biology or on blood sugar in healthy people.

The slug `thao-nhat` is excluded by hand: *đái tháo nhạt* (diabetes insipidus) shares "đái tháo" with *đái tháo đường* but is an unrelated disease of the kidneys and hormones.

The result is **1,197 candidate URLs**. Six were unreachable or duplicates of an article already seen under another address (the crawler skips a page whose title it already holds), so **1,191 pages** were downloaded and checked: 944 with a strong keyword, 247 with only a weak one.

Manners. ♦`robots.txt` allows these paths. The crawler names itself as a student research crawler, waits one second between requests, retries failures with growing pauses, saves each page the moment it is processed, and skips pages already saved, so an interrupted run resumes where it stopped.

3.3 The accent trap, and a second filter

Dropping accents in slugs hides a trap. Vietnamese has many words that differ only by accent, and without accents they collapse together:

- `tieu-duong` is *tiểu đường* (diabetes) but also the start of *tiểu dương vật* (a urology question about a child);
- `dai-duong` is *đái đường* (diabetes) but also *đại dương* (ocean), as in *cá ngừ đại dương* (ocean tuna);
- `duong` also hides *dưỡng ẩm* (moisturising) and *dương tính* (a positive test).

Those pages pass the slug test and have nothing to do with diabetes. So every downloaded page is re-

checked on its real text, **with accents**, where the words cannot be confused. Two counts are taken: the *diabetes count* (*tiểu đường*, *đái tháo đường*, *ĐTĐ*) and the *related count* (*đường huyết*, *insulin*, *HbA1c*, *biến chứng*…). A page is kept if:

1. it is an official **disease page**, which Vinmec itself classed as diabetes; or
2. its **title or summary** mentions diabetes and the text mentions it twice (the title is the author's own statement of the topic); or
3. its slug had a **strong** keyword and the text mentions diabetes at least three times (the accent collisions mention it zero times); or
4. its slug had only a **weak** keyword, and the text has at least five diabetes mentions *and* five related terms (an "Insulin là gì?" article that names the hormone thirty times and diabetes once is biology, not disease).

The thresholds are loose where the evidence is strong (the title) and strict where a wrong page is likely (a weak keyword).

Table 3. Candidate pages kept and rejected, by strength of the slug keyword.

SLUG KEYWORD	KEPT	REJECTED	CANDIDATES
strong	934	10	944
weak	162	85	247
all	1,096	95	1,191

Table 4. Some rejected pages with their diabetes and related counts. The first four are accent collisions.

TITLE	SLUG	DIABETES	RELATED
Trẻ 19 tháng đi tiểu dương vật không cử động cần làm gì?	strong	0	0
Vô tình hít phải Heroin nên đi thử nước tiểu dương tính có phải không?	strong	0	0
Kết quả xét nghiệm nước tiểu dương tính với Amphetamine và Methamphet…	strong	0	0
Thành phần dinh dưỡng trong cá ngừ đại dương	strong	2	0
Chế độ ăn cho trẻ thiếu men glucose 6 và mang gen dương tính với bệnh…	weak	0	6
Bé 5 ngày tuổi bị tụt đường huyết phải điều trị trong bao lâu?	weak	0	7

Only 10 of the 944 strong-keyword pages were rejected, all accent collisions. A third of the weak-keyword pages (85 of 247) were rejected: they concern insulin, blood sugar or tests in general. The filter is imperfect: the article *"Bài thuốc từ bí đao chữa bệnh tiểu đường"* is about diabetes but too short to qualify. Losing a few short pages is cheap; admitting off-topic ones would put false facts into the graph.

3.4 Parsing a page: keep the headings

The crawler does not store raw HTML, which is mostly menus, adverts, booking forms, footers and scripts. It uses Vinmec's page structure instead:

- **articles:** the body is the element with class `entry`; title, date, author and summary come from the page's structured metadata (`application/ld+json`);
- **disease pages:** each section is an element with class `item_detial_sick` holding a heading and a body.

The body is cut into **sections**, and each section saves its **heading**, its **paragraphs** and its **bullet points** separately.

THE MOST IMPORTANT DESIGN CHOICE

> Keeping the heading. It says what kind of information the text below it holds: the bullet "Mệt mỏi" (fatigue) means one thing under *Triệu chứng* (symptoms) and another under *Tác dụng phụ của thuốc* (side effects). Part 4.3 builds everything on this.

The tail of each article is cut off too (the booking text *"Để đặt lịch khám tại viện, Quý khách vui lòng bấm số HOTLINE..."*, the app advert, the *"Xem thêm:"* links). Left in, every article would "mention" the same booking words and the related-article links would drag in vocabulary from other topics. Figure 2 shows a stored page.

```
title: Tiểu đường tuýp 2: Nguyên nhân, triệu chứng, chẩn đoán và điều trị
url:   https://www.vinmec.com/vie/benh/tieu-duong-tuyp-2-3042     type: disease

## Tổng quan bệnh Tiểu đường tuýp 2           (4 paragraphs, 0 bullets)
## Nguyên nhân bệnh Tiểu đường tuýp 2         (1 paragraph,  0 bullets)
## Triệu chứng bệnh Tiểu đường tuýp 2         (3 paragraphs, 4 bullets)
     • Ăn nhiều
     • Uống nhiều (hay khát nước)
     • Tiểu nhiều (do đường trong nước tiểu cao, gây lợi niệu thẩm thấu)
     • Gầy nhiều (gầy sút cân): bệnh nhân tiểu đường tuýp 2 thường có ...
## Biến chứng cấp tính                        (1 paragraph,  0 bullets)
## Biến chứng mạn tính                        (0 paragraphs, 6 bullets)
     • Nhìn mờ (do biến chứng võng mạc, đục thủy tinh thể)
     • Tê bì dị cảm ở bàn chân (biến chứng thần kinh)
     ...
## Đối tượng nguy cơ bệnh Tiểu đường tuýp 2    ...
## Các biện pháp chẩn đoán bệnh Tiểu đường tuýp 2  ...
## Các biện pháp điều trị bệnh Tiểu đường tuýp 2   ...
```

Figure 2. A stored page: the type 2 disease page as a list of sections, each with heading, paragraphs and bullets (abridged).

3.5 The corpus at a glance

Table 5. Size of the collected corpus.

QUANTITY	VALUE
Candidate URLs from the sitemap	1,197
Pages downloaded and checked	1,191
Pages kept as diabetes articles	1,096
Pages rejected	95
Disease pages / articles kept	4 / 1,092
Words in the kept pages	1,130,914
Median page	1044 words
Median diabetes mentions per page	16
Pages stamped 2024 (of those with a date)	1,083 of 1,092

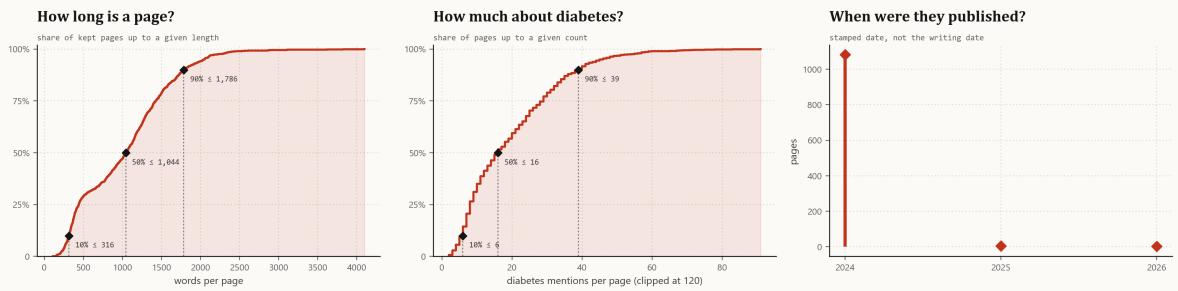


Figure 3. Left: length of the kept pages. Middle: how often a page mentions diabetes. Right: publication year.

A typical article has about a thousand words, enough to cover several aspects under separate headings. Half the kept pages mention diabetes 16 times or more and only one in ten fewer than 6, so the filter kept pages about the disease, not pages that brush against it. Nearly every dated page (1,083 of 1,092) carries a 2024 date. That is the date of Vinmec's re-publication of its archive, not of writing, so true article age is unknown, which Part 11 counts as a limitation.

PART 4

From Prose to Triples

4.1 Triples, nodes and edges

A knowledge graph is a set of **triples**: the thing a fact is about (**subject**), the kind of link (**relation**) and the other thing (**object**). The type 2 page's lists of symptoms and chronic complications become, for instance:

SUBJECT	RELATION	OBJECT
Đái tháo đường type 2	HAS_SYMPTOM	Khát nhiều
Đái tháo đường type 2	HAS_SYMPTOM	Tiểu nhiều
Đái tháo đường type 2	HAS_COMPLICATION	Bệnh võng mạc
Bệnh võng mạc	AFFECTS_ORGAN	Mắt

This beats the paragraph because every fact has the same predictable shape. A program needs no Vietnamese grammar: it looks for subject "type 2" and relation HAS_COMPLICATION. The ambiguity of language is resolved once, when the fact is made, not every time a question is asked.

Draw the things as nodes and the facts as arrows and you get a graph; each node also carries a **type** (Symptom, Drug…). Figure 4 shows seven facts written by hand. Questions become walks: "which organ does type 2 damage?" goes from *type 2* along HAS_COMPLICATION to *Bệnh thận đái tháo đường*, then along AFFECTS_ORGAN to *Thận*. No single fact holds the answer; the node in the middle joins two. The lecture calls that a **two-hop** query, against a **one-hop** query that reads only the arrows on one node.

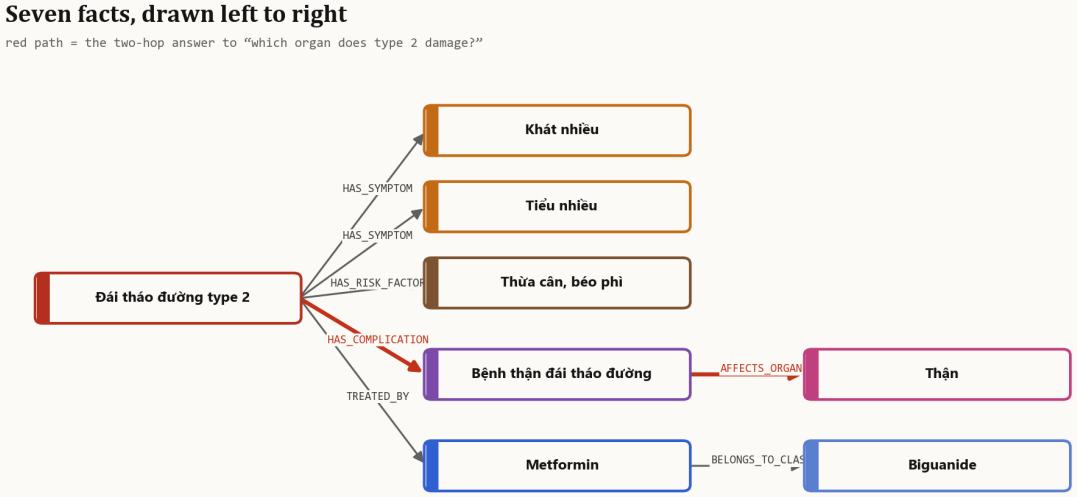


Figure 4. Seven hand-written diabetes facts. The kidney is two hops from type 2: no single fact links them.

The real graph has the same shape but is built from 1,096 articles automatically. The hard part is not storing it; it is extracting **correct** facts from free Vietnamese text without anyone reading every article. The rest of this part explains how.

4.2 Schema: entity types and relation types

The schema says which kinds of things exist and which kinds of facts may join them. It was designed from the questions of Table 2 and from the way Vinmec articles are organised.

Entity types. ♦There are ten (Table 6):

- five mirror the standard sections of a disease article: symptoms, risk factors, complications, tests, treatments;
- treatments split into `Drug` and `Lifestyle`, because the questions differ ("which drug class?" makes sense only for drugs);
- `Food` stands alone because diet is one of the most discussed topics;
- `DrugClass` and `Organ` are not extracted; they group drugs and locate complications in the body (Part 4.3.9);
- the five `DiabetesType` nodes are the subjects of nearly every extracted fact.

Table 6. Entity types: vocabulary entries, aliases, entities in the graph, examples.

TYPE	VOCAB	ALIASES	IN GRAPH	EXAMPLES
`DiabetesType`	–	–	5	type 2, gestational
`Symptom`	16	56	16	Khát nhiều, Sụt cân
`RiskFactor`	17	52	16	Kháng insulin, Thừa cân
`Complication`	19	52	18	Bệnh võng mạc, Đột quỵ
`Test`	10	35	10	HbA1c, OGTT
`Drug`	22	42	22	Metformin, Insulin
`DrugClass`	–	–	9	Biguanide, Ức chế SGLT2
`Food`	16	66	16	Rau xanh, Rượu bia
`Lifestyle`	6	17	6	Tập thể dục đều đặn
`Organ`	–	–	9	Thận, Mắt
total	106	320	127	

Almost every vocabulary entry reached the graph. The missing risk factor and complication are explained in Part 10.

Relation types. ♦Table 7 lists the twelve relations, the types they join and their fact counts. The source column separates facts extracted from articles (`vinmec`) from classifications added by hand (`curated`).

Table 7. Relation types in the graph.

RELATION	FROM → TO	SOURCE	FACTS
`HAS_COMPLICATION`	DiabetesType → Complication	vinmec	52
`TREATED_BY`	DiabetesType → Drug	vinmec	46
`HAS_SYMPTOM`	DiabetesType → Symptom	vinmec	45
`HAS_RISK_FACTOR`	DiabetesType → RiskFactor	vinmec	42
`RECOMMENDS_FOOD`	DiabetesType → Food	vinmec	35
`DIAGNOSED_BY`	DiabetesType → Test	vinmec	30
`AVOIDS_FOOD`	DiabetesType → Food	vinmec	26
`BELONGS_TO_CLASS`	Drug → DrugClass	curated	22
`MANAGED_BY`	DiabetesType → Lifestyle	vinmec	19
`AFFECTS_ORGAN`	Complication → Organ	curated	12
`HAS_SIDE_EFFECT`	Drug → Complication	vinmec	6
`SUBTYPE_OF`	DiabetesType → DiabetesType	curated	4

Arrow direction is a modelling choice: `HAS_SYMPTOM` could just as well read `SYMPTOM_OF` the other way.

Here facts point *away* from the diabetes types and on to classifications, and Part 6.5 shows why that matters for graph measures. Figure 5 draws the schema: each ribbon is a kind of question the atlas can answer, and a missing ribbon (Food to Complication, say) is a question it cannot.

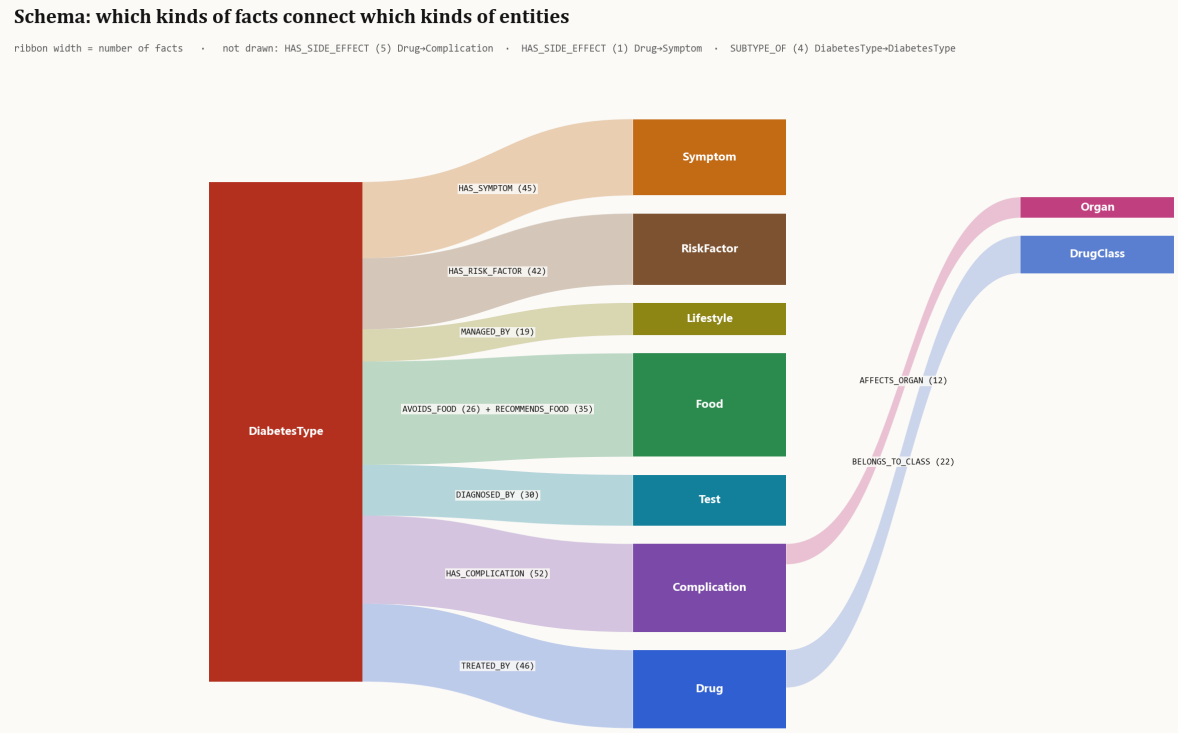


Figure 5. The schema as stored in Neo4j. Ribbon width is the number of facts of that kind.

4.3 Extraction method

Why a bullet is not a node

The simplest plan is to turn every bullet under *Triệu chứng* into a Symptom node. On the type 2 page that yields "Ăn nhiều" and "Uống nhiều (hay khát nước)", which looks fine. But a bullet under *Điều trị* on the type 1 page reads:

> *"Kiểm soát đường huyết: đối với tuýp 1 thì kiểm soát bằng insulin ngoại sinh là chủ yếu. Dùng theo phác đồ, tuân thủ giờ tiêm và liều lượng tránh bị tụt đường huyết. Các loại insulin gồm có…"*

As a node, sixty words are useless: no article will repeat them, so the node joins nothing and no question matches it. Even short bullets differ (*uống nhiều*, *khát nước*, *hay khát*), splitting "diabetes causes thirst" into weak pieces that never meet.

So the project decides **in advance** what the graph may contain, gives each thing one clean name, and searches the text for every way it can be written. For every section of every page the extractor then answers three questions:

1. **Which diabetes is the page about?** That is the subject.
2. **What may this section contain?** The heading decides.
3. **Which known concepts does it mention?** Those are the objects.

Each fact on a page is one **vote**; votes from all pages are counted; a fact enters the graph only when enough different pages agree. Figure 6 follows one real section.

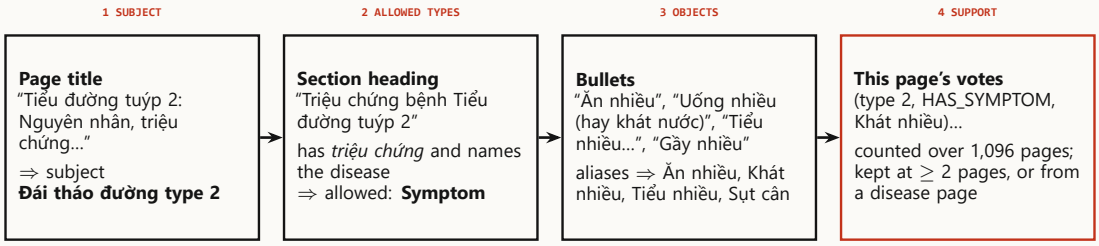


Figure 6. Extracting symptom facts from the symptom section of the type 2 disease page.

Vocabulary and entity normalisation

The vocabulary (`kg/vocab.py`) holds 106 concepts. Each has a clean **canonical name** (the node), a **type** and a list of **aliases**, the spellings the articles use (320 in all):

```
VOCAB = {
    "Khát nhiều":    ("Symptom", ["khát nhiều", "uống nhiều", "khát nước", "hay khát", "khô miệng"]),
    "Tê bì tay chân": ("Symptom", ["tê bì", "tê tay", "tê chân", "dị cảm", "ngứa ran", "kiến bò"]),
    "Kháng insulin": ("RiskFactor", ["kháng insulin", "đề kháng insulin"]),
    "Bệnh võng mạc": ("Complication", ["võng mạc", "biến chứng mắt", "mù lòa"]),
    "HbA1c":         ("Test", ["hba1c", "hemoglobin a1c", "a1c"]),
    "Metformin":     ("Drug", ["metformin", "glucophage"]),
    "Rau xanh":      ("Food", ["rau xanh", "rau lá xanh", "rau củ", "bông cải", "súp lơ"]),
    ...
}
```

Mapping many spellings to one node is **entity normalisation**, and it makes evidence add up: *uống nhiều* in one article and *khát nước* in another vote for the same fact, and the graph sees two articles agreeing. Brand names are aliases too (*Glucophage*, *Diamicron*, *Januvia*, *Jardiance*, *Lantus*), as Vietnamese articles often name drugs by brand.

The list was written by hand from what the articles discuss. The price is plain (a concept not on the list can never be found, see Part 11); so is the gain (every node is clean, meaningful and correctly typed).

Finding an alias in the text

All aliases of an entity merge into one regular expression, for *Tiểu nhiều*:

```
(?<!\w)(đi tiểu thường xuyên|tiểu thường xuyên|đi tiểu nhiều|tiểu nhiều|tiểu đêm)(?!\w)
```

Three details matter.

- **Whole words only.** Vietnamese separates syllables with spaces, so many aliases are one short syllable that also lives inside longer words: *lê* (pear) inside *lên* ("lên cân", gaining weight), *bia* (beer) inside *biên*. The `(?<!\w)` and `(?!\w)` parts demand non-letters on both sides.
- **Longest alias first.** Tried the other way round, "tiểu thường xuyên" would win and the more precise wording would never be used.
- **Accents stay.** Text is lower-cased so "HbA1C" equals "hba1c", but accents are never stripped: that would reintroduce the *tiểu đường* / *tiểu dương* collision.

Not solved: an alias that is a whole word with a second meaning. The fruit *táo* (apple) matches inside *táo bón* (constipation). The section rule below keeps this rare, because a food alias counts only in diet sections.

The section rule: the heading decides

A vocabulary word appearing in an article is not a fact about it. Real cases:

- *mệt mỏi* (fatigue) in a diet article: "ăn sáng đầy đủ giúp bạn đỡ mệt mỏi", which does not make fatigue a symptom;
- *tăng huyết áp* (high blood pressure) in drug paragraphs, as a condition not to combine with the drug;
- *insulin*, named in nearly every article while explaining blood sugar, not because it is the treatment discussed.

If every match counted, the graph would claim dozens of "symptoms" that are merely words that happened to occur. So a match counts only when the **section heading** says the section is about that kind of information. Hospital articles are organised by exactly such headings (Table 8).

Table 8. The section rule: heading keywords and the entity types allowed below them.

HEADING CONTAINS	ALLOWED	RELATION
triệu chứng, dấu hiệu, biểu hiện, nhận biết	Symptom	HAS_SYMPTOM
nguyên nhân, nguy cơ, đối tượng, yếu tố, ai dễ	RiskFactor	HAS_RISK_FACTOR
biến chứng, nguy hiểm, hậu quả, ảnh hưởng	Complication	HAS_COMPLICATION
chẩn đoán, xét nghiệm, phát hiện, chỉ số, kiểm tra, theo dõi	Test	DIAGNOSED_BY
điều trị, thuốc, insulin, kiểm soát	Drug, Lifestyle	TREATED_BY, MANAGED_BY
phòng ngừa, phòng tránh, ngăn ngừa, lối sống, lưu ý	Lifestyle	MANAGED_BY
ăn, thực phẩm, dinh dưỡng, thực đơn, món, uống, kiêng, tránh	Food	RECOMMENDS_FOOD / AVOIDS_FOOD

A heading with none of these words (*Tổng quan*, the overview) yields nothing. That is deliberate: overviews mention everything briefly, and the project prefers a missed fact to an invented one.

An extra gate for symptoms and risk factors. ♦The first rule had a flaw that the quality check of Part 8 exposed. A heading can say *nguyên nhân* or *nguy cơ* and still not be about diabetes:

- *"Một số nguyên nhân gây ngứa bàn tay và bàn chân"* lists causes of **itching**;
- *"Yếu tố nguy cơ gây tai biến mạch máu não ở người tiểu đường"* lists risks of a **stroke** in people who already have diabetes;
- *"Làm gì để giảm thiểu nguy cơ tim mạch…"* is advice on **reducing** heart risk.

Reading those bullets as risk factors *of diabetes* gave wrong facts. A symptom or risk factor always belongs to *something*, and the heading names it. So for these two types only, the heading must also pass `about_disease()`:

```
DISEASE_TERMS = ["tiểu đường", "đái tháo đường", "đtđ", "tuýp", "type", "típ", "bệnh"]
OTHER_TOPIC   = ["biến chứng", "thận", "võng mạc", "mắt", "thần kinh", "bàn chân", "tim mạch",
                 "ngứa", "gout", "da", "răng", "nhiễm toan", "ceton", "hạ đường huyết",
                 "thai nhi", "trẻ sinh ra", "tai biến", "đột quỵ", "mạch vành", "hôn mê"]
REDUCE_VERBS  = ["giảm", "phòng", "tránh", "ngừa", "hạn chế", "kiểm soát"]

def about_disease(heading):
    """Generic heading ("Triệu chứng", "Ai dễ bị?") or one that names diabetes, and no other topic."""
    h = re.sub(r"^[\d\.\s]+", "", heading.lower())          # drop numbering such as "2.1."
    if any(t in h for t in OTHER_TOPIC) or any(v + " nguy cơ" in h for v in REDUCE_VERBS):
        return False
    return any(t in h for t in DISEASE_TERMS) or len(h.split()) <= 5

def allowed_labels(heading):
    h = heading.lower()
    labels = set()
    for keys, labs in SECTION_RULES:                        # Table 7
        if any(k in h for k in keys):
            labels |= labs
    if not about_disease(heading):
        labels -= {"Symptom", "RiskFactor"}
    return labels
```

In words: the heading must name diabetes or be short and generic (nothing else it could be about), must not name another topic, and must not be about reducing a risk. Table 9 applies it to real headings.

Table 9. The section rule on real headings.

HEADING	ABOUT DIABETES?	ALLOWED
Triệu chứng bệnh Tiểu đường tuýp 2	yes	Symptom
Đối tượng nguy cơ bệnh Tiểu đường tuýp 1	yes	RiskFactor
3. Ai dễ bị tiền tiểu đường?	yes	RiskFactor
Các biện pháp điều trị bệnh Tiểu đường tuýp 1	yes	Drug, Lifestyle
2.2. Một số thực phẩm mẹ bầu mắc bệnh tiểu đường thai kỳ cần tránh	yes	Food
3. Một số nguyên nhân gây ngứa bàn tay và bàn chân	no	–
2. Yếu tố nguy cơ gây tai biến mạch máu não ở người tiểu đường	no	–
4. Làm gì để giảm thiểu nguy cơ tim mạch ở bệnh nhân tiểu đường?	no	–
Biến chứng mạn tính	no (not needed)	Complication

Which diabetes is the subject?

Diabetes is several diseases. Type 1 is autoimmune (the body destroys its insulin-making cells, usually from youth); type 2 develops slowly in adults and tracks weight and lifestyle; gestational diabetes appears in pregnancy; prediabetes precedes type 2. Their symptoms, causes and treatments overlap without being identical: "autoimmune destruction of beta cells" causes type 1 only, "pre-eclampsia" complicates gestational diabetes only. Pinning every fact to one "diabetes" node would erase this.

Each page therefore takes its subject from its **title**, the author's own short statement of the topic, which is more reliable than counting words in the body (a type 2 article often mentions type 1 for comparison). The title is searched for *tuýp 1 / type 1 / típ 1*, *tuýp 2 / type 2*, *thai kỳ / mang thai / mẹ bầu*, *tiền đái tháo đường / tiền tiểu đường* in turn; otherwise the subject is diabetes in general.

The shortcut leaves a visible mark (Figure 7): most titles name no type, so the general node receives most facts even though many of those articles are really about type 2. It was chosen on purpose: a type 2 fact on the general node is merely *less specific*; on the type 1 node it would be *false*.

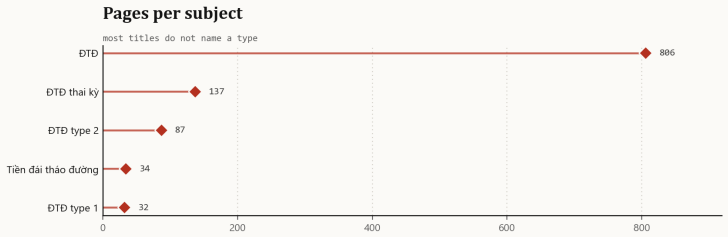


Figure 7. Pages per subject. Most titles do not name a specific type.

Counting evidence: one page is not enough

Any rule can still slip on one page: an odd heading, a passing mention, a bullet that lists something for contrast. Trust every page and every slip becomes a fact. The defence is **agreement between independent pages**: for each fact the extractor keeps the *set* of pages that voted for it (a page repeating "khát nhiều" five times counts once), and the size of the set is the fact's **support**. A worked example with three type 2 pages:

PAGE	VOTES
A	type 2 HAS_SYMPTOM Khát nhiều; type 2 HAS_SYMPTOM Mệt mỏi
B	type 2 HAS_SYMPTOM Khát nhiều
C	type 2 HAS_SYMPTOM Khát nhiều; type 2 HAS_SYMPTOM Run tay

Khát nhiều has support 3 and is kept; *Mệt mỏi* and *Run tay* have support 1 and are dropped. Two exceptions, each with a reason:

- a fact from an official **disease page** is accepted at support 1: those pages are short hospital-reviewed summaries with exactly the standard headings;
- a **side effect** needs support 3, because it is read from single sentences and is noisier.

```
for (h, rel, t), urls in support.items():                # urls = set of pages voting for the fact
    need = MIN_SUPPORT_SIDE_EFFECT if rel == "HAS_SIDE_EFFECT" else MIN_SUPPORT   # 3 or 2
    if len(urls) >= need or (h, rel, t) in from_disease:
        triples.append((h, rel, t, len(urls), "vinmec", " ".join(sorted(urls)[:5])))
```

The support number outlives the filter: it is stored on the fact in Neo4j, so queries can rank by how well documented a fact is and the website lets readers hide thinly supported ones.

Foods: recommended or to avoid

Foods have two relations, `RECOMMENDS_FOOD` and `AVOIDS_FOOD`. The first version chose from the words near the food (*nên ăn* = recommended; *kiêng*, *tránh*, *hạn chế*, *không nên* = avoid) and failed instructively: diet articles mix both kinds of advice in one place. The heading *"Thực phẩm mẹ bầu cần tránh và nên ăn"* holds both, and one sentence can say "hạn chế đồ ngọt, thay vào đó nên ăn rau xanh". The output included nonsense like *gestational diabetes AVOIDS green vegetables*.

The final rule makes good-or-bad a **property of the food**, written once in the vocabulary:

- **avoid:** sweets, soft drinks, white rice and refined starch, very sweet fruit (durian, ripe mango, longan, jackfruit), alcohol, fried and processed meat;
- **recommended:** the rest.

The text then only shows that the food is discussed in a diet section for that type. This matches standard nutrition advice and what the articles say almost always; the cost is that nuance ("fruit is fine in moderation") cannot be represented.

Drug side effects

Side effects appear anywhere (drug sections, complication sections, general articles), so the section rule cannot find them. They are read from **one sentence**: it must contain a drug, a symptom or complication, and a cue (*tác dụng phụ*, *tác dụng không mong muốn* or *gây*):

"Insulin có thể **gây hạ đường huyết** nếu tiêm quá liều" ⇒ (Insulin, `HAS_SIDE_EFFECT`, Hạ đường huyết)

Sentences with *thiếu* (lack of) or *không đủ* (not enough) are skipped: "*thiếu* insulin gây nhiễm toan ceton" has a drug, a complication and "gây", yet says a *lack* of insulin causes ketoacidosis, the opposite of a side effect. Part 10 shows the filter catches most such sentences, not all.

Background facts added by hand

Some knowledge is general medical classification that articles take for granted. These facts are added by hand and tagged `source = curated` so they can always be told from extracted ones:

- each drug and its **class** (22 facts: Metformin → Biguanide, Empagliflozin → Ức chế SGLT2…);
- each complication and the **organ** it affects (12 facts);
- each diabetes type as a **subtype** of diabetes in general (4 facts);
- the diagnostic **thresholds** on four test nodes, from the Vietnamese Ministry of Health 2020 guideline and the ADA criteria that Vinmec also quotes: HbA1c $\geq$ 6.5%; fasting glucose $\geq$ 7.0 mmol/L; 2-hour glucose in the tolerance test $\geq$ 11.1 mmol/L; random glucose $\geq$ 11.1 mmol/L with symptoms.

These few facts unlock the best questions. Without "retinopathy affects the eye" the graph could say type 2 causes retinopathy but not answer "which organs does type 2 damage?".

4.4 Output files

The extractor writes four CSV files:

Table 10. Files in `dataset/processed/`.

FILE	ONE ROW IS	WHY IT EXISTS
`entities.csv`	an entity: name, type, threshold	becomes the nodes
`triples.csv`	a fact: subject, relation, object, support, source, up to five example URLs	becomes the relationships
`articles.csv`	a page: URL, title, type, date, subject	lets every fact be traced to its articles
`mentions.csv`	a pair (page, entity) for every entity mentioned anywhere on the page	a looser signal of what each article talks about

`triples.csv` uses the lecture's `subject, relation, object` form plus three columns:

```
subject,relation,object,support,source,example_urls
Đái tháo đường type 2,HAS_SYMPTOM,Khát nhiều,4,vinmec,https://www.vinmec.com/vie/bai-viet/...
Đái tháo đường type 2,TREATED_BY,Metformin,11,vinmec,https://www.vinmec.com/vie/bai-viet/...
Metformin,BELONGS_TO_CLASS,Biguanide,0,curated,
Bệnh võng mạc,AFFECTS_ORGAN,Mắt,0,curated,
```

Table 11. The best-supported extracted fact of each relation type.

SUBJECT	RELATION	OBJECT	SUPPORT
Đái tháo đường	`TREATED_BY`	Insulin	143
Đái tháo đường	`MANAGED_BY`	Tập thể dục đều đặn	114
Đái tháo đường	`HAS_COMPLICATION`	Bệnh tim mạch	56
Đái tháo đường	`RECOMMENDS_FOOD`	Ngũ cốc nguyên hạt	54
Đái tháo đường	`AVOIDS_FOOD`	Rượu bia	48
Đái tháo đường	`DIAGNOSED_BY`	HbA1c	42
Insulin	`HAS_SIDE_EFFECT`	Hạ đường huyết	37
Đái tháo đường	`HAS_RISK_FACTOR`	Thừa cân, béo phì	27
Đái tháo đường	`HAS_SYMPTOM`	Mệt mỏi	25

4.5 Scorecard against the lecture

Table 12. Lecture requirements (Mini Project 3, Exercise 1) and what the graph reaches.

REQUIREMENT	LECTURE MINIMUM	THIS PROJECT
Entities	$\geq$ 50	127
Relationships	$\geq$ 100	339
Relationship types	$\geq$ 5	12
Entity types	$\geq$ 3	10
Cypher queries	$\geq$ 5 (one-hop, two-hop)	15
Triples as CSV	`subject, relation, object`	`triples.csv` + support, source, URLs
Visualisation	NetworkX	ring, radial, arc, ribbon and chart views
Graph retrieval	1–2 hops from an entity	`graph_facts(entity, hops)`

PART 5

Under the Hood

5.1 Project layout

Python covers the crawler, extractor, loader, API and notebook; JavaScript covers the website. Every stage reads files or the database and can be re-run alone.

Table 13. Layout of `graph_01/`.

PATH	ROLE
`kg/crawl.py`	sitemap → candidates → download → relevance filter → `dataset/raw/`; resumable
`kg/vocab.py`	106 entities, 320 aliases, section rules, food polarity, curated classes, organs, thresholds
`kg/extract.py`	pages → votes → support filter → `dataset/processed/*.csv`
`kg/load.py`	CSV → Neo4j (wipes and reloads; idempotent)
`kg/retriever.py`	`graph_facts(entity, hops)`, bounded retrieval
`kg/visualize.py`, `kg/atlas.py`	the drawings and the plotting theme
`kg/evaluate.py`	draws the random sample for the quality check
`kg/queries.cypher`	the 15 example queries for Neo4j Browser
`notebook/01_diabetes_kg.ipynb`	explains and runs every stage with all figures
`api/`, `web/`	FastAPI read-only service; React website
`evaluation/manual_audit.csv`	the 50 sampled facts and their judgement

5.2 Neo4j

Why a graph database

The in-memory NetworkX graph of the first examples vanishes when the program ends and must be rebuilt by every program that wants it. The real system needs knowledge that is **stored permanently**, **shared** by several programs and **answered quickly**.

A plain relational table of three columns could hold the triples, but graph questions are about *paths*, and in a table every step of a path is a join of the table with itself: queries grow long and slow. A **graph database** keeps each node with direct references to its neighbours, so following an arrow is a jump, not a search. **Neo4j** is the most widely used one; its language **Cypher** draws a path in ASCII-art. Neo4j 5 runs in a Docker container with its data in the project folder.

Data model

```
(:Entity:Symptom {name})                      every entity: label Entity + its own type
(:Entity:Test {name, threshold})
(:DiabetesType)-[:HAS_SYMPTOM {support, source, example_urls}]->(:Symptom)
(:Article {url, title, page_type, date})-[:ABOUT]->(:DiabetesType)
(:Article)-[:MENTIONS]->(:Entity)
```

- **Two labels per entity.** The type label lets a query ask for "all symptoms". The shared `Entity` label carries one rule for all (names are unique, so there can never be two "Metformin" nodes) and lets "everything connected to Metformin" be written without knowing neighbours' types.
- **Evidence on the relationships.** `support`, `source` and `example_urls` sit on each fact, so one query returns the answer, how well documented it is and which articles state it.
- **Articles are nodes.** Each of the 1,096 articles links by `ABOUT` to its type and by `MENTIONS` to every entity it mentions (10,083 links). The thread from clean facts back to raw text answers "where does this come from?". Since articles and `MENTIONS` are not knowledge about diabetes, knowledge queries stay on `Entity` nodes.

Loading

```
CREATE CONSTRAINT entity_name IF NOT EXISTS FOR (e:Entity) REQUIRE e.name IS UNIQUE;

UNWIND $rows AS r
MERGE (e:Entity {name: r.name}) SET e:Symptom;          -- one query per entity type

UNWIND $rows AS r
MATCH (h:Entity {name: r.subject}), (t:Entity {name: r.object})
MERGE (h)-[x:HAS_SYMPTOM]->(t)                          -- one query per relation type
SET x.support = r.support, x.source = r.source, x.example_urls = r.example_urls;
```

- **`MERGE`, not `CREATE`**: look first, create only if missing, so reloading never duplicates.

- **`UNWIND` sends a batch**: ten thousand mention links in one request instead of ten thousand round trips; everything loads in seconds.
- **The uniqueness constraint is an index**: a query starting at "the node called Metformin" jumps straight to it.
- **Relation types go into the query text**: Cypher cannot take them as parameters, so the loader builds one query per type from the fixed list of twelve.

After loading: 1,223 nodes (127 entities, 1,096 articles) and 11,518 relationships (339 facts, 10,083 mentions, 1,096 `ABOUT`).

5.3 Cypher

A Cypher query draws the *shape* of a path, with round brackets for nodes and square brackets for relationships:

```
MATCH (t:DiabetesType {name: 'Đái tháo đường type 2'})-[r:HAS_SYMPTOM]->(s:Symptom)
RETURN s.name, r.support
```

Read left to right: find a `DiabetesType` named "type 2", follow an outgoing `HAS_SYMPTOM` arrow to a `Symptom`, return its name and the arrow's support. Neo4j jumps to "type 2" through the name index and looks only at the arrows on that node, never at the rest of the database, so the work grows with the arrows followed, not with database size. The fifteen queries of `kg/queries.cypher` are run in Part 7.

5.4 Bounded retrieval

In a full question-answering system (the lecture's KG + RAG architecture) the graph hands a language model a small set of relevant, reliable facts. `kg/retriever.py` implements the lecture's `graph_facts` on Neo4j:

```
ONE_HOP = """
MATCH (e:Entity {name: $name})-[r]-(n:Entity)
RETURN startNode(r).name AS s, type(r) AS rel, endNode(r).name AS o, r.support AS support
ORDER BY support DESC"""
TWO_HOP = """
MATCH (e:Entity {name: $name})-[r1]-(m:Entity)-[r2]-(n:Entity) WHERE n <> e
RETURN startNode(r2).name AS s, type(r2) AS rel, endNode(r2).name AS o, r2.support AS support
ORDER BY support DESC"""

def graph_facts(driver, entity, hops=1, limit=30):
    facts, seen = [], set()
    with driver.session() as s:
        for q in [ONE_HOP] + ([TWO_HOP] if hops >= 2 else []):
            for r in s.run(q, name=entity):
                key = (r["s"], r["rel"], r["o"])
                if key not in seen:
                    seen.add(key)
                    facts.append(f"{r['s']} {r['rel']} {r['o']}.")
    return facts[:limit]
```

It starts at the entity in the question, collects facts within one or two steps, writes each as a short sentence, sorts by support and returns at most 30:

```
$ python kg/retriever.py "Hạ đường huyết"
Đái tháo đường HAS_COMPLICATION Hạ đường huyết.
Insulin HAS_SIDE_EFFECT Hạ đường huyết.
Đái tháo đường thai kỳ HAS_COMPLICATION Hạ đường huyết.
Metformin HAS_SIDE_EFFECT Hạ đường huyết.
Đái tháo đường type 2 HAS_COMPLICATION Hạ đường huyết.
```

Why the limits? Almost everything in a graph is connected to everything within a few steps, especially through hubs. Figure 8 shows it: one hop around hypoglycaemia gives 5 precise facts, two hops give **230**, because a first-step neighbour is the general diabetes node that touches the whole graph. Handing 230 facts to a language model for a hypoglycaemia question buries the relevant five. This is the lecture's "graph retrieval too broad" problem (§23.4), cured as it recommends: start at the entity, stay within one or two hops, rank and cut.

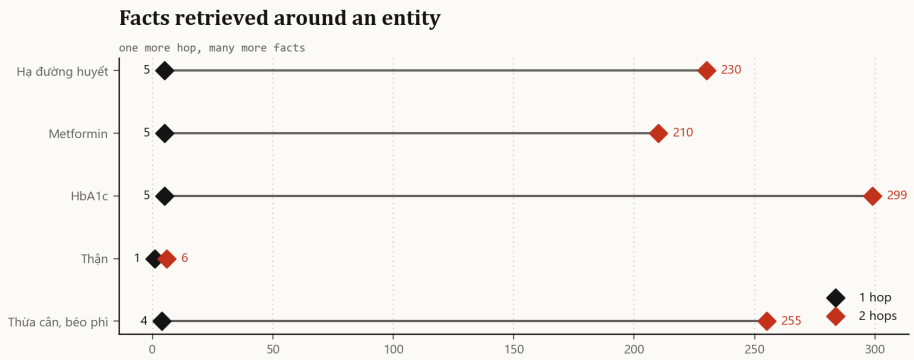
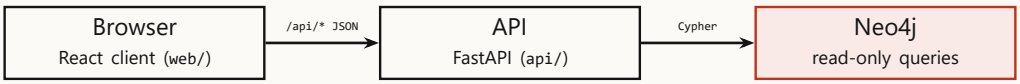


Figure 8. Facts returned around five entities at one and at two hops.

5.5 The website

Architecture

Cypher is powerful but not for casual readers, so the graph gets a face. It has two parts: an **API** (`api/`, Python, FastAPI) that turns simple HTTP requests into fixed Cypher and returns JSON, and a **web client** (`web/`, React 18, Vite, hand-written CSS) that people see.



Why an API in the middle rather than the browser talking to Neo4j? *Security*: the password never leaves the server and the browser can only run the eight fixed read-only queries. *Stability*: the site depends on a small documented interface, so the schema can change without breaking it. *Simplicity*: every request is a plain URL (`/entities/Metformin`), testable in a browser and documented automatically by FastAPI at `/docs`.

API

Table 14. API endpoints.

ENDPOINT	RETURNS
`GET /health`	whether the API and the database are up
`GET /stats`	size of the graph: entities per type, relation counts, facts, articles
`GET /entities?q=&type=`	search suggestions: names that contain the text, prefix matches first, then best connected
`GET /entities/{name}`	one entity with all its facts (both directions), filtered by minimum support
`GET /entities/{name}/articles`	the latest articles that mention the entity
`GET /evidence`	the articles that state one fact
`POST /symptom-check`	diabetes types ranked for a list of symptoms
`GET /paths?source=&target=`	shortest chains of facts between two entities (up to four steps)

Every query is parameterised (user input travels as `$name`, never pasted into the query text), every response has a declared Pydantic shape that validates and documents it, and errors become plain messages: an unknown entity gives `404 No entity named…`, a stopped database `503`, which the site shows as a banner telling the reader what to start. The entity endpoint:

```
@app.get("/entities/{name}", response_model=EntityDetail)
def entity(name: str, min_support: int = Query(0, ge=0)):
    rows = db.read(f"""
        MATCH (e:Entity {{name: $name}})
        RETURN e.name AS name, {TYPE.format(v='e')} AS type, e.threshold AS threshold,
               COUNT {{ (:Article)-[:MENTIONS|ABOUT]->(e) }} AS mentioned_in""", name=name)
    if not rows:
        raise HTTPException(404, f"No entity named '{name}'.")
    facts = db.read(f"""
```

```
        MATCH (e:Entity {{name: $name}})-[r]-(n:Entity)
        WHERE r.source = 'curated' OR r.support >= $min_support
        WITH startNode(r) AS s, r, endNode(r) AS o
        RETURN s.name AS subject, {TYPE.format(v='s')} AS subject_type, type(r) AS relation,
               o.name AS object, {TYPE.format(v='o')} AS object_type,
               coalesce(r.support, 0) AS support, r.source AS source
        ORDER BY support DESC LIMIT $cap""", name=name, min_support=min_support, cap=config.MAX_FACTS)
    return {**rows[0], "facts": facts}
```

The path finder uses Neo4j's `allShortestPaths` over entity nodes only (otherwise any two entities would be linked through an article that mentions both), up to four steps, and tells the site whether each fact points forward or backward along the path so it can say "*is a risk factor for*" instead of "*has the risk factor*".

Web client

The client is called **Glyco Atlas** and presented as a field guide: a fixed side rail with the wordmark, numbered chapters and a search box; serif headings over ruled ledgers instead of cards; one diamond-marked colour per entity type. Five pages, each with its own address so any view can be shared (`#/entity/Metformin`, `#/paths?from=…&to=…`):

1. **Home**: search with suggestions, graph size, a start point for each diabetes type, a clickable legend of entry kinds, and "how this atlas was compiled";
2. **Entity**: all facts about one entity under plain headings, each with its support and a *Cite* button listing the articles behind it; a drawing of the neighbourhood; the latest articles that mention it;
3. **Diabetes types**: that page for each type, in tabs;
4. **Symptom finder**: pick symptoms, see which types the articles tie to them;
5. **Trace a link**: pick two entities, see the shortest chains of facts between them.

Several choices come from the evaluation, not from taste:

- **Evidence is always visible**: support on every fact, sources one click away, hand-added facts marked "reference".
- **The reader picks the evidence level**: a selector hides facts stated by fewer than 2, 3, 5 or 10 articles (default 2), handing the precision/recall trade-off of Part 10 to the user.
- **Relations in plain words**: `HAS_COMPLICATION` reads "Complications" from the disease and "Complication of" from the complication.
- **The disclaimer sits where it matters**, on the symptom finder.
- **Every state is handled**: loading, empty results, unknown entity, stopped API or database. The search is a standard accessible combobox, and the layout adapts to phones.

Part 9 walks through the site.

PART 6

Seeing the Graph

A graph of 127 entities and 339 facts cannot be grasped from tables. Force-directed layouts turn it into a hairball, so every drawing here uses a layout chosen for readability: a ring, a radial view, ribbons, an arc diagram. All are drawn from the same CSV files that were loaded into Neo4j, so they show exactly the same facts.

6.1 Size of the graph

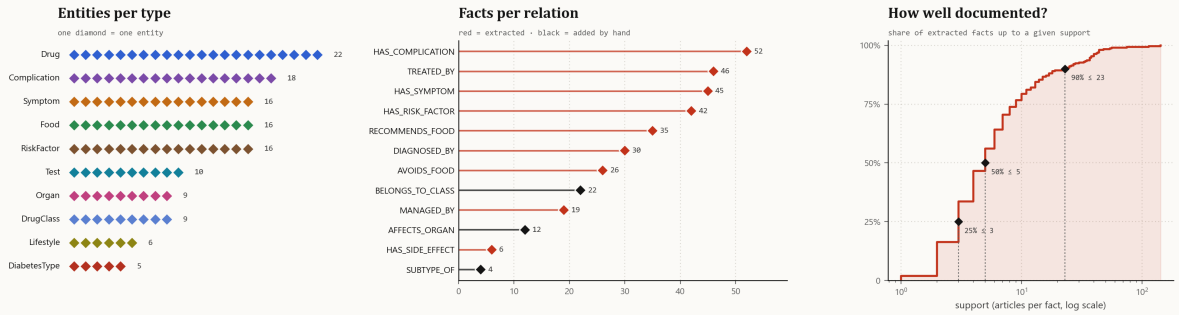


Figure 9. Left: entities per type. Middle: facts per relation. Right: support of the extracted facts, on a log scale.

- **Entities per type.** Drugs, complications, risk factors, symptoms and foods each have 16 to 22 entities, about the size of their vocabulary, so nearly every concept was found with enough support.
- **Support.** The telling chart. The median fact has support 5, but a few are repeated by dozens: “diabetes is treated with insulin” by 143 articles, “managed by regular exercise” by 114, “can cause cardiovascular disease” by 56. The core message is in nearly every article while specifics (which class a newer drug belongs to, which complications are pregnancy-only) appear in a few. Support is therefore both a noise filter and a measure of **how central a fact is** to Vinmec’s way of writing about diabetes.

6.2 The whole atlas on a ring

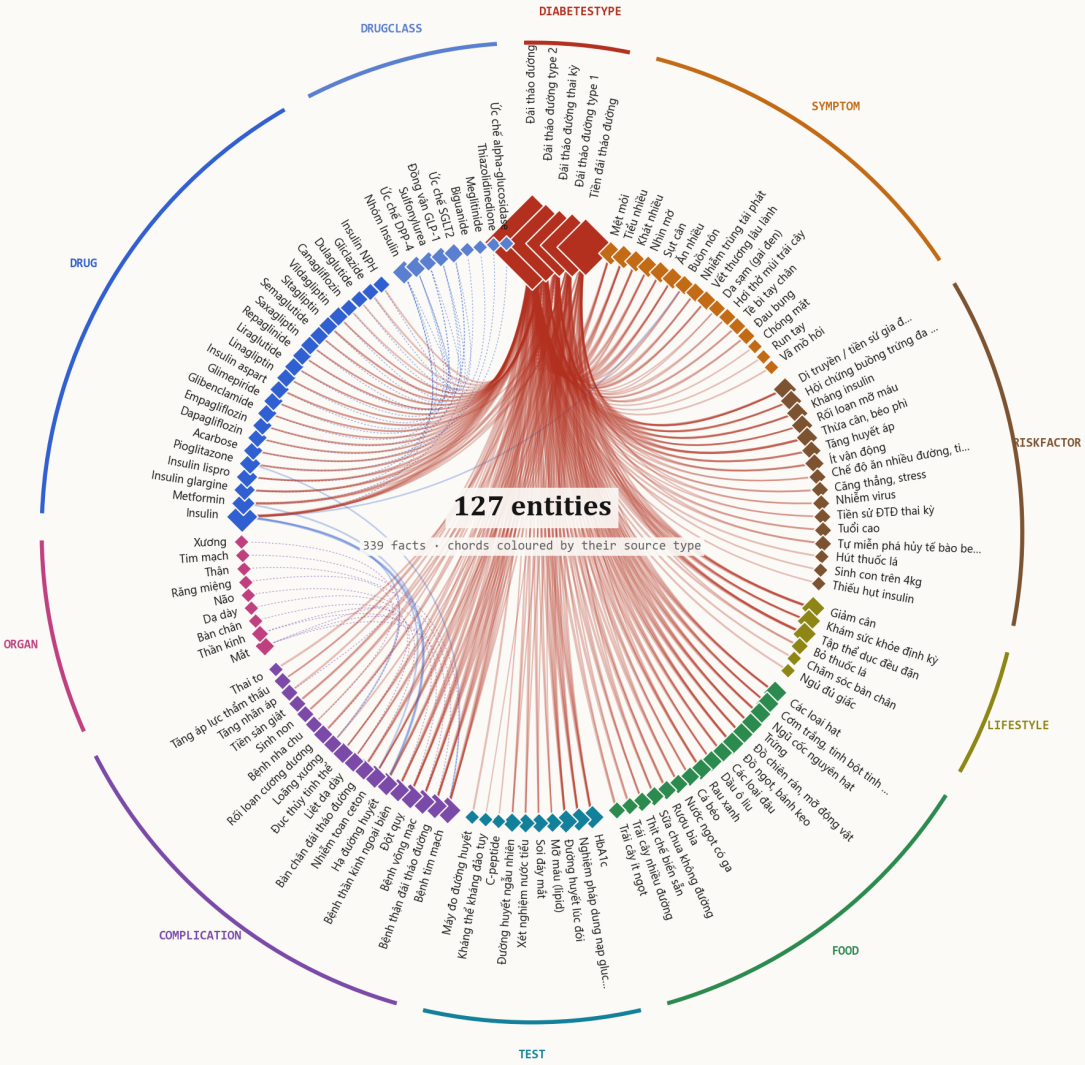


Figure 10. All 339 facts. Entities sit on a ring grouped by type (coloured arcs); each fact is a chord in the colour of the type it starts from, thicker for more support, dashed when added by hand.

Figure 10 makes the structure plain. The five diabetes types at the top are the hubs: nearly every chord starts there and fans out to symptoms, risk factors, foods, tests, complications and drugs. Dashed chords (drug → class, complication → organ) are the hand-added ones, and they stay in the lower-left, away from the hubs.

The graph is **sparse**: of all arrows that could join 127 entities, about 2% exist (density 0.021). Real knowledge looks like this, and it is why following one node's arrows touches only a small part of the graph.

6.3 Close-ups

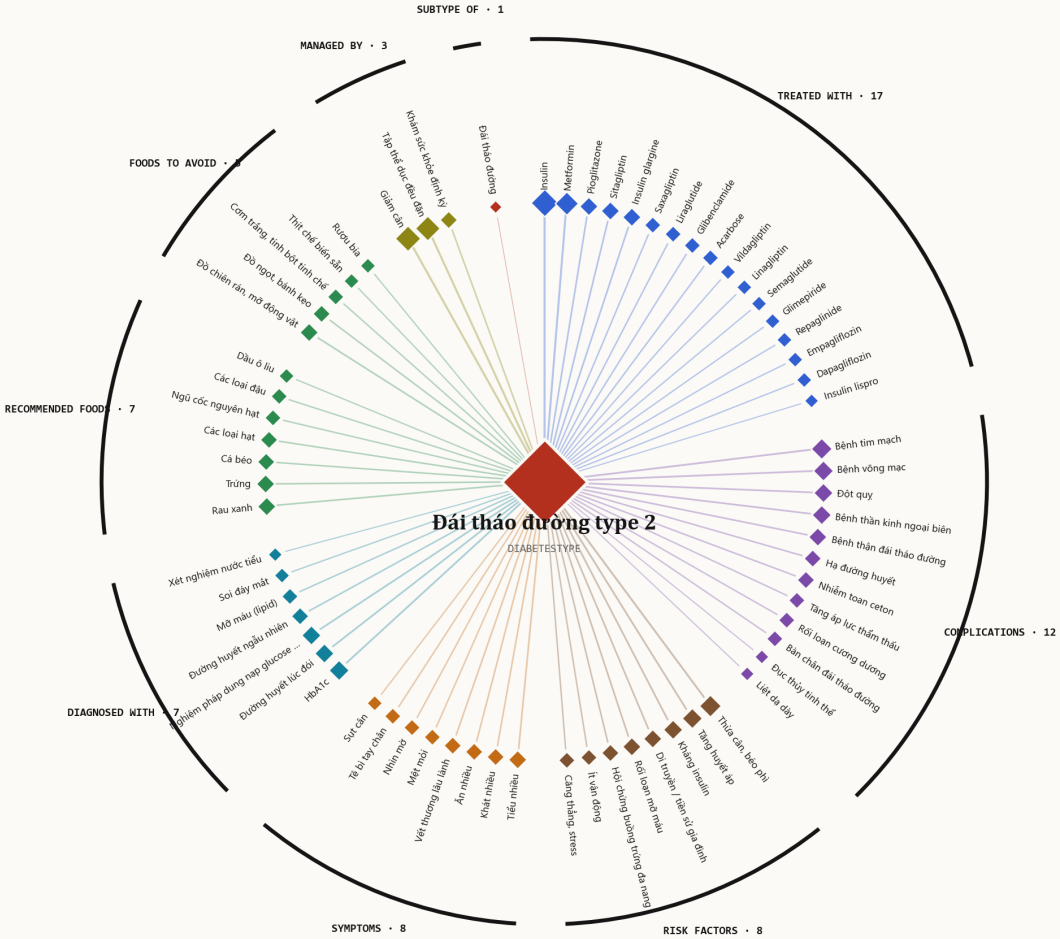


Figure 11. Everything the graph knows directly about type 2 diabetes, grouped by relation. Diamond size shows support.

Figure 11 is a one-hop query made visible: the type 2 node and 17 drugs, 12 complications, 8 risk factors, 8 symptoms, 7 tests, 12 foods and 3 lifestyle measures.

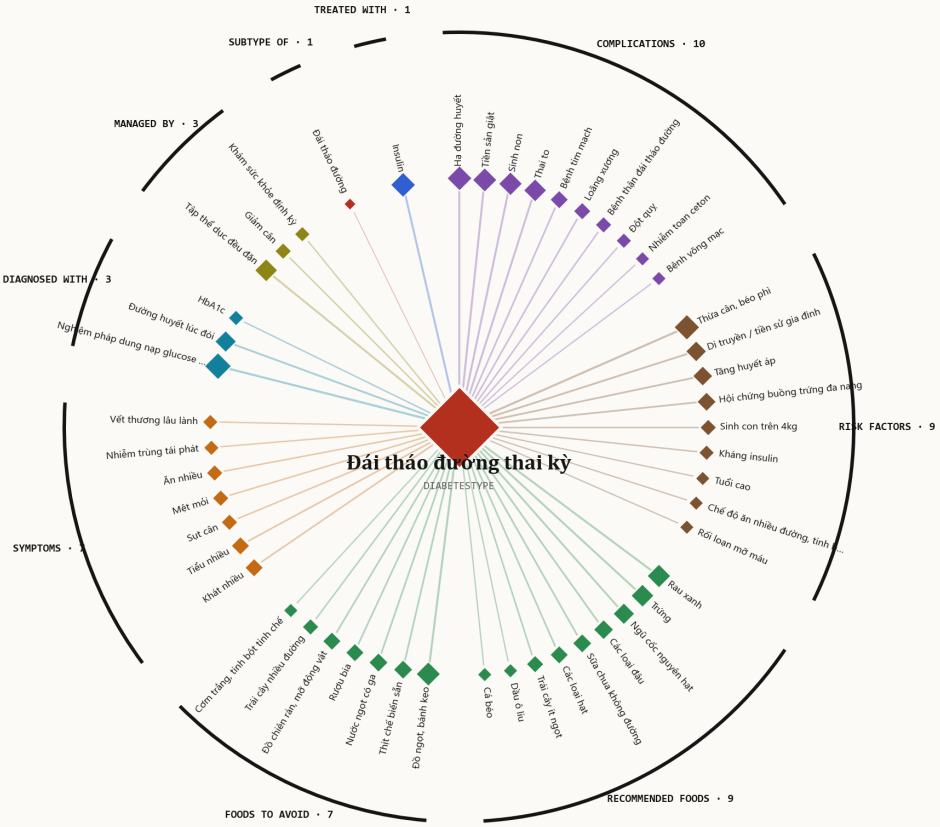


Figure 12. The one-hop neighbourhood of gestational diabetes.

Gestational diabetes (Figure 12) differs from type 2: a single drug, insulin (most oral drugs are avoided in pregnancy), and complications of its own, pre-eclampsia, premature birth and a very large baby. The graph picked this up from the articles unprompted.

Type 2 treatment as an arc diagram

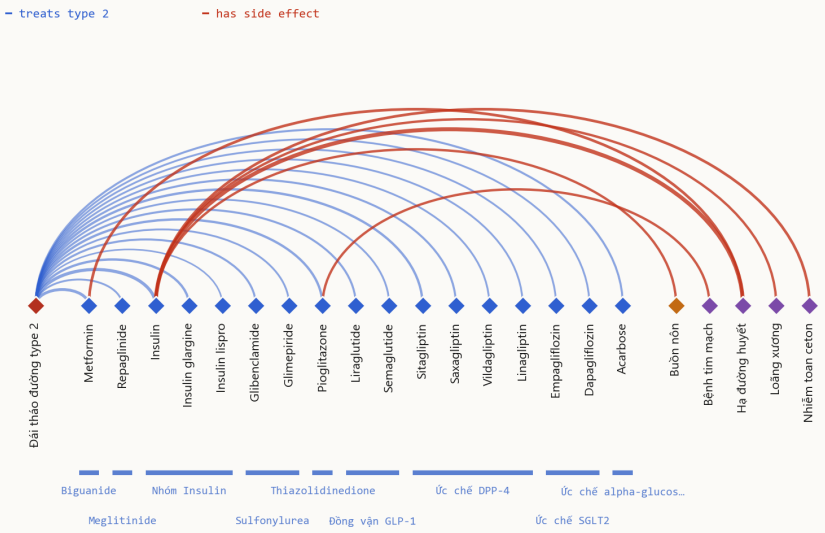


Figure 13. Type 2 treatment as an arc diagram. Blue arcs: type 2 is treated by a drug. Red arcs: a drug has a side effect. Brackets below group drugs by class.

Figure 13 mixes extracted and hand-added facts. The brackets make drug families visible (three DPP-4 inhibitors, three SGLT2 inhibitors, the insulin family), which is how doctors think of them, and the red arcs show the best-known risk of treatment, hypoglycaemia, reached from both insulin and Metformin.

6.4 Type by type

If giving each type its own subject (Part 4.3) works, the types should differ. Figure 14 counts the supporting articles for each complication and symptom per type.



Figure 14. Supporting articles per diabetes type for the most common complications (left) and symptoms (right).

Two things show. The *general* column is the largest, as most titles name no type. And *gestational* diabetes has its own signature: hypoglycaemia (of mother and newborn), pre-eclampsia, premature birth and a large baby, pregnancy complications the other types lack, while types 1 and 2 share the classic long-term damage to eyes, kidneys, nerves and heart. That is medically correct, so the subject rule captures a real difference.

6.5 What matters most

"Important" means different things for a node, and two usual measures disagree here, which is instructive.

- **Degree**: how many facts an entity takes part in; it sees only the node's own arrows.
- **PageRank**, Google's original ranking method: a node matters if *important* nodes point to it. Picture a walker who follows a random outgoing arrow, but 15% of the time jumps to a random node; after a long walk, PageRank is the share of time spent on each node.

What happens here. All arrows point away from the diabetes types: type → drug → class, type → complication → organ. The walker drifts down these chains and gets stranded at their ends, which have no outgoing arrows, until the next random jump. Directed PageRank therefore ranks *Nhóm Insulin*, *Ức chế DPP-4* and *Mắt* (eye) first: not what the articles are about, just where paths *end*.

Arrow direction is our own modelling choice ("type 2 `HAS_SYMPTOM` thirst" could be "thirst `SYMPTOM_OF` type 2"), and a measure that flips when a relation is renamed measures nothing useful. So PageRank is also computed **ignoring direction**. Then an entity matters if it is well connected to well-connected entities: the types first, then insulin, cardiovascular disease, stroke, diabetic kidney disease, retinopathy and Metformin. Figure 15 sets the two side by side. The lesson: every graph measure answers a specific question, and you must check it is the one you meant.

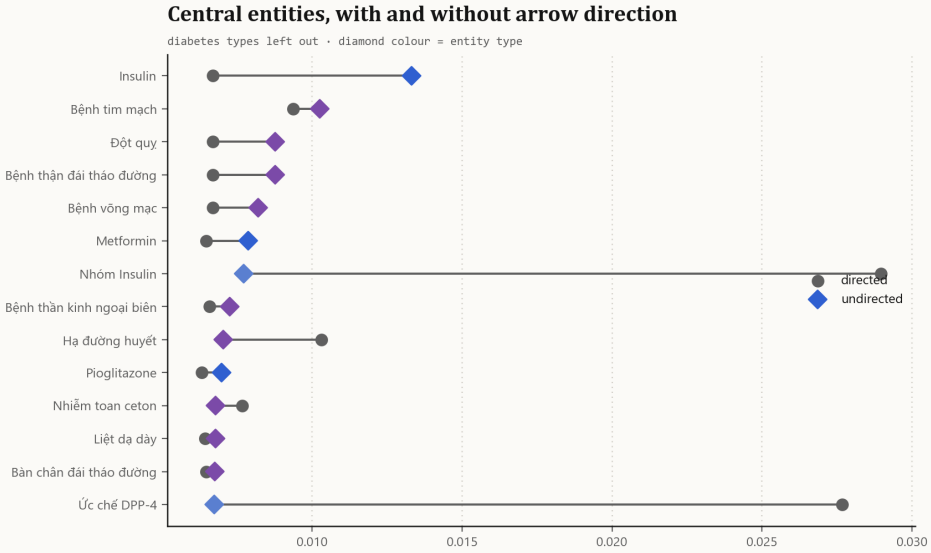


Figure 15. The 14 most central non-type entities. Grey dots: directed PageRank. Diamonds: undirected PageRank.

A third view ignores facts and counts, for each entity, the articles (of 1,096) that mention it anywhere via `MENTIONS` (Figure 16). That measures what the site *talks about*, which can differ from how connected an entity is among facts: something can be named in many articles in passing, say while explaining blood sugar, and still sit in few facts under a proper heading.

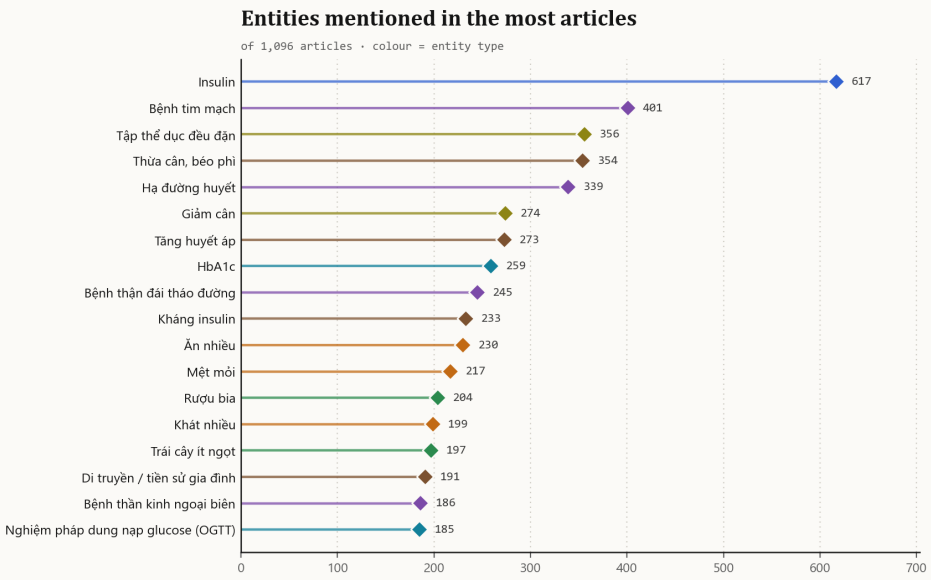


Figure 16. Entities mentioned in the most articles.

PART 7

Putting the Graph to Work

The lecture asks that an intelligent system be evaluated systematically, not judged on whether it "gives an answer". Parts 7 and 8 do it two ways: **what it can answer** (the questions of Table 2 run as Cypher and checked for sense) and **how correct its facts are** (a random sample checked by hand).

The queries come from `kg/queries.cypher`, fifteen in all (the lecture requires five). Each is shown with its live result.

7.1 One hop: everything about Metformin

```
MATCH (e:Entity {name: 'Metformin'})-[r]-(n:Entity)
RETURN startNode(r).name AS subject, type(r) AS relation, endNode(r).name AS object,
       r.support AS support, r.source AS source
ORDER BY support DESC
```

Table 15. One-hop neighbourhood of Metformin.

SUBJECT	RELATION	OBJECT	SUPPORT	SOURCE
Đái tháo đường	TREATED_BY	Metformin	37	vinmec
Đái tháo đường type 2	TREATED_BY	Metformin	11	vinmec
Tiền đái tháo đường	TREATED_BY	Metformin	8	vinmec
Metformin	HAS_SIDE_EFFECT	Hạ đường huyết	5	vinmec
Metformin	BELONGS_TO_CLASS	Biguanide	0	curated

The pattern `-[r]-` has no arrowhead, so it matches both directions: facts pointing *to* Metformin (diabetes *is treated with* it) and the one pointing *from* it (it *belongs to* the biguanide class). Support shows solidity: 37 articles say diabetes is treated with Metformin, 11 name type 2, 8 mention prediabetes (where it delays onset of type 2), 5 mention hypoglycaemia as a risk. The class has support 0 because it was added by hand.

7.2 Two hops: which organs can type 2 damage?

```
MATCH (t:DiabetesType {name: 'Đái tháo đường type 2'})-[r:HAS_COMPLICATION]->(c)
      -[:AFFECTS_ORGAN]->(o:Organ)
RETURN o.name AS organ, collect(c.name) AS via_complications, sum(r.support) AS evidence
ORDER BY evidence DESC
```

Table 16. Organs reached from type 2 diabetes in two hops.

ORGAN	VIA COMPLICATIONS	EVIDENCE
Thần kinh	Rối loạn cương dương, Bệnh thần kinh ngoại biên	9
Mắt	Đục thủy tinh thể, Bệnh võng mạc	8
Tim mạch	Bệnh tim mạch	8
Não	Đột quỵ	6
Thận	Bệnh thận đái tháo đường	5
Bàn chân	Bàn chân đái tháo đường	3
Dạ dày	Liệt dạ dày	1

No article says "type 2 damages the nerves" as one fact. The answer exists because an extracted fact (type 2 *can cause* peripheral neuropathy) meets a hand-added one (neuropathy *affects* the nerves) at the complication node, and `collect()` gathers the complications behind each organ. The result matches the textbook list: nerves, eyes, heart and vessels, brain (stroke), kidneys, feet, stomach.

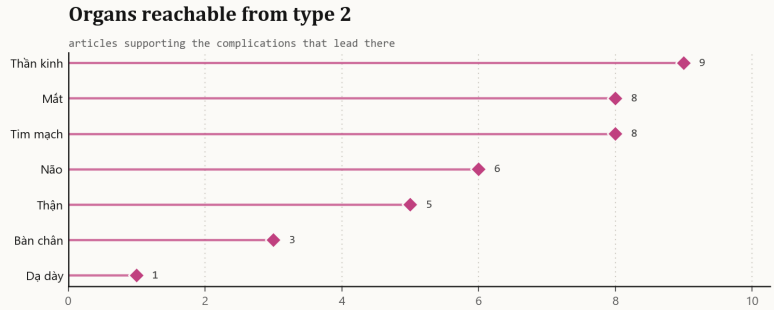


Figure 17. The two-hop answer of Table 16 as a chart.

7.3 Type 1 against type 2

```
MATCH (s:Symptom)
OPTIONAL MATCH (:DiabetesType {name: 'Đái tháo đường type 1'})-[a:HAS_SYMPTOM]->(s)
OPTIONAL MATCH (:DiabetesType {name: 'Đái tháo đường type 2'})-[b:HAS_SYMPTOM]->(s)
WITH s, a, b WHERE a IS NOT NULL OR b IS NOT NULL
RETURN s.name, a.support AS type1, b.support AS type2
```

Table 17. Symptoms of type 1 and type 2, with supporting articles per type.

SYMPTOM	TYPE 1	TYPE 2	FOUND IN
Khát nhiều	7	4	both
Mệt mỏi	6	3	both
Nhìn mờ	5	3	both
Sụt cân	7	2	both
Tiểu nhiều	6	5	both
Ăn nhiều	6	4	both
Buồn nôn	4		type 1 only
Hơi thở mùi trái cây	3		type 1 only
Nhiễm trùng tái phát	5		type 1 only
Đau bụng	5		type 1 only
Tê bì tay chân		3	type 2 only
Vết thương lâu lành		4	type 2 only

- The six **shared** symptoms (thirst, frequent urination, eating a lot, weight loss, fatigue, blurred vision) come from high blood sugar itself, whatever the cause, so sharing is right.
- Most **type 1 only** symptoms (nausea, abdominal pain, fruity breath) are signs of **ketoacidosis**, the acid build-up when the body has almost no insulin, typical of type 1 at discovery. (Recurrent infections also appear only for type 1: a general high-sugar sign that type 2 articles did not list under a symptom heading.)
- The **type 2 only** symptoms (slow-healing wounds, numb hands and feet) arrive after years of moderately high sugar, matching type 2's slow, quiet course.

The graph reproduced a real medical difference without being told.

7.4 A symptom finder

```
WITH ['Khát nhiều', 'Tiểu nhiều', 'Sụt cân', 'Nhìn mờ'] AS input
MATCH (t:DiabetesType)-[r:HAS_SYMPTOM]->(s:Symptom) WHERE s.name IN input
RETURN t.name AS type, count(s) AS matched, collect(s.name) AS symptoms,
       sum(r.support) AS evidence
ORDER BY matched DESC, evidence DESC
```

Table 18. Types ranked for {thirst, frequent urination, weight loss, blurred vision}.

TYPE	MATCHED	SYMPTOMS	EVIDENCE
Đái tháo đường	4	Khát nhiều, Nhìn mờ, Sụt cân, Tiểu nhiều	67
Đái tháo đường type 1	4	Khát nhiều, Nhìn mờ, Sụt cân, Tiểu nhiều	25
Đái tháo đường type 2	4	Khát nhiều, Nhìn mờ, Sụt cân, Tiểu nhiều	14
Đái tháo đường thai kỳ	3	Khát nhiều, Sụt cân, Tiểu nhiều	16
Tiền đái tháo đường	2	Nhìn mờ, Tiểu nhiều	7

Types are ranked by how many given symptoms they have; ties go to the larger total support. General diabetes, type 1 and type 2 match all four, and general leads only because most articles are general. This shows both the use and the limit: the graph says which types the articles associate with these symptoms and how strongly, but classic symptoms are shared by every type, so it cannot tell them apart. That takes blood tests, which is the next query.

7.5 Diagnosis: tests and thresholds

```
MATCH (t:DiabetesType)-[r:DIAGNOSED_BY]->(x:Test)
RETURN x.name AS test, x.threshold AS threshold, sum(r.support) AS evidence
ORDER BY evidence DESC
```

Table 19. Diagnostic tests: hand-added thresholds, extracted evidence.

TEST	THRESHOLD	EVIDENCE
HbA1c	$\geq$ 6.5%	67
Nghiệm pháp dung nạp glucose (OGTT)	2 giờ $\geq$ 11.1 mmol/L (200 mg/dL)	59
Đường huyết lúc đói	$\geq$ 7.0 mmol/L (126 mg/dL)	51
Xét nghiệm nước tiểu	–	14
Soi đáy mắt	–	11
Đường huyết ngẫu nhiên	$\geq$ 11.1 mmol/L kèm triệu chứng	11
Máy đo đường huyết	–	10
Mỡ máu (lipid)	–	7
C-peptide	–	2
Kháng thể kháng đảo tụy	–	1

Both kinds of knowledge meet here: “diabetes is diagnosed with HbA1c” was extracted, “6.5% or more” is a property of the test node added from the official guideline. The three best-evidenced tests (HbA1c, the glucose tolerance test, fasting glucose) are exactly the three standard criteria. Thinly evidenced tests are specialised: the islet-antibody test and C-peptide help tell type 1 from 2. The antibody test links to type 1 (from the type 1 disease page), but C-peptide only to general diabetes, since its two articles name no type in the title.

7.6 Diet in gestational diabetes

Table 20. Foods discussed in diet sections of gestational-diabetes articles.

ADVICE	FOOD	ARTICLES
avoid	Đồ ngọt, bánh kẹo	14
avoid	Thịt chế biến sẵn	7
avoid	Nước ngọt có ga	7
avoid	Rượu bia	6
avoid	Đồ chiên rán, mỡ động vật	6
avoid	Trái cây nhiều đường	4
avoid	Cơm trắng, tinh bột tinh chế	2
recommended	Rau xanh	13
recommended	Trứng	12
recommended	Ngũ cốc nguyên hạt	10
recommended	Các loại đậu	8
recommended	Sữa chua không đường	7
recommended	Các loại hạt	6
recommended	Trái cây ít ngọt	5
recommended	Cá béo	2
recommended	Dầu ô liu	2

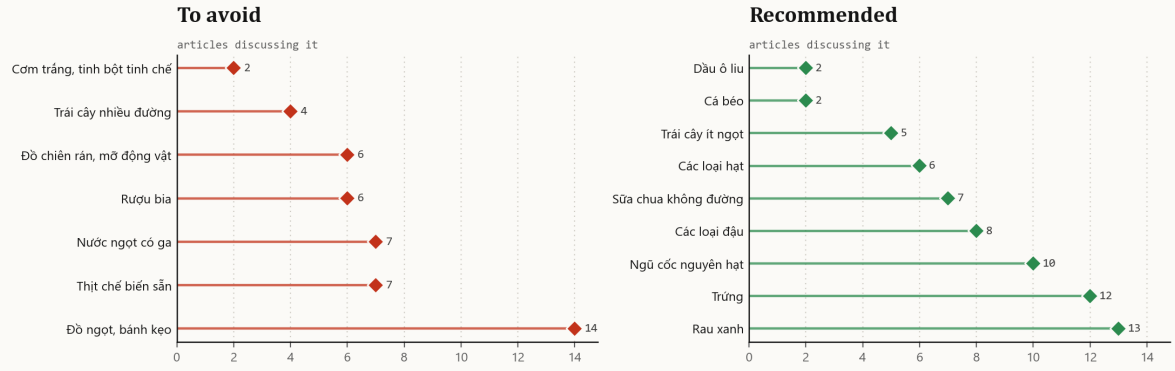


Figure 18. The same data as lollipops: foods to avoid (red) and recommended (green).

Sweets lead the foods to avoid; green vegetables, eggs and whole grains lead the recommended, in line with usual advice for pregnancy. The avoid/recommended split comes from the fixed food list; what the articles add is *which* foods are discussed for this type and how often.

7.7 Provenance: where does a fact come from?

```
MATCH (:DiabetesType {name: 'Đái tháo đường type 2'})-[r:HAS_COMPLICATION]->
      (:Complication {name: 'Bệnh thận đái tháo đường'})
UNWIND r.example_urls AS url
MATCH (a:Article {url: url})
RETURN a.title
```

Table 21. Articles stating that type 2 diabetes can cause diabetic kidney disease.

ARTICLE TITLE
Bệnh tiểu đường type 2 có nguy hiểm không?
Bệnh tiểu đường type 2: Nguyên nhân, biến chứng và phòng bệnh
Các biến chứng nguy hiểm của bệnh đái tháo đường type 2
Cách tự nhiên điều trị bệnh tiểu đường type 2 với những thực phẩm hàng ngày
Triệu chứng sớm của đái tháo đường type 2

Each fact keeps up to five example URLs; joining them with article nodes yields real Vinmec titles anyone can open. That is what makes a graph trustworthy for health: a language model may say something fluent and wrong with no way to check, while every answer here leads back to its source.

7.8 Written about together

Table 22. Complication pairs that share the most articles.

COMPLICATION	COMPLICATION	ARTICLES
Bệnh tim mạch	Hạ đường huyết	170
Bệnh thận đái tháo đường	Bệnh tim mạch	163
Bệnh tim mạch	Đột quỵ	139
Bệnh thận đái tháo đường	Hạ đường huyết	116
Bệnh thần kinh ngoại biên	Bệnh tim mạch	110
Bệnh thận đái tháo đường	Bệnh võng mạc	107
Bệnh thần kinh ngoại biên	Bệnh thận đái tháo đường	100
Bệnh tim mạch	Bệnh võng mạc	98

`MENTIONS` links give a looser signal: which topics share articles. Co-appearance is not a fact about diabetes, but it shows how Vinmec writes. Cardiovascular disease and hypoglycaemia lead because each is in a huge share of all articles; raw co-occurrence always favours common topics. Finding *unusually* frequent pairs would need each count compared with what the two topics' popularity predicts.

PART 8

How Far to Trust It

8.1 What was measured, and how

The measure is **precision**: of the facts in the graph, what share are correct? It is the share a user actually feels, since every wrong fact is stated with confidence. The procedure (`kg/evaluate.py`):

1. draw 50 extracted facts at random with a fixed seed (42), so the same sample can be drawn again;
2. show each with its article title and a sentence from a section the extractor used;
3. judge it by reading **all** its supporting articles, recording `correct` and `error_type` in `evaluation/manual_audit.csv`.

A fact is correct when the Vinmec articles state it for that type of diabetes. That checks against the *source*, not clinical guidelines, which would be a different and far costlier review. Random, because hand-picking would favour easy facts and flatter the result; fifty, because each fact means reading several articles, and fifty is enough to estimate precision usefully and to expose the main error kinds.

Table 23. The first 12 of the 50 sampled facts. Support is the value at audit time, before the stricter heading rule of Part 10.

#	SUBJECT	RELATION	OBJECT	SUPPORT	CORRECT
1	Tiền đái tháo đường	HAS_COMPLICATION	Bệnh thần kinh ngoại biên	5	yes
2	Đái tháo đường thai kỳ	AVOIDS_FOOD	Thịt chế biến sẵn	7	yes
3	Đái tháo đường thai kỳ	HAS_RISK_FACTOR	Chế độ ăn nhiều đường, tinh b…	2	yes
4	Đái tháo đường	HAS_RISK_FACTOR	Rối loạn mỡ máu	16	**no**
5	Tiền đái tháo đường	HAS_RISK_FACTOR	Tuổi cao	2	yes
6	Đái tháo đường	HAS_COMPLICATION	Tăng nhãn áp	9	yes
7	Đái tháo đường type 2	DIAGNOSED_BY	Đường huyết ngẫu nhiên	4	yes
8	Đái tháo đường	TREATED_BY	Saxagliptin	5	yes
9	Đái tháo đường type 1	DIAGNOSED_BY	Xét nghiệm nước tiểu	2	yes
10	Đái tháo đường type 2	TREATED_BY	Glimepiride	2	yes
11	Đái tháo đường	MANAGED_BY	Ngủ đủ giấc	7	yes
12	Đái tháo đường thai kỳ	AVOIDS_FOOD	Trái cây nhiều đường	4	yes

8.2 Result

Table 24. Precision on the 50 sampled facts, by relation.

RELATION	SAMPLED	CORRECT
AVOIDS_FOOD	7	7
DIAGNOSED_BY	6	6
HAS_COMPLICATION	7	7
HAS_RISK_FACTOR	11	10
HAS_SYMPTOM	7	6
MANAGED_BY	2	2
RECOMMENDS_FOOD	3	3
TREATED_BY	7	7
all	50	48



Figure 19. The sample at a glance and the likely range for the whole graph.

48 of 50 facts are correct, a precision of 0.96. Both errors are in symptoms and risk factors, the two relations that lean hardest on the heading; every complication, test, drug and food fact in the sample was right.

How precise is "0.96"? Another random 50 might hold one error or four. The standard calculation for a proportion on a small sample (the Wilson interval) puts the precision of the whole graph between **0.87 and 0.99**: roughly nine in ten or better, not an exact figure.

8.3 How much evidence the facts carry

Table 25. Extracted facts left when a minimum number of supporting articles is required.

MINIMUM SUPPORT	FACTS	SHARE
≥ 1	301	100%
≥ 2	295	98%
≥ 3	252	84%
≥ 5	161	53%
≥ 10	70	23%
≥ 20	32	11%

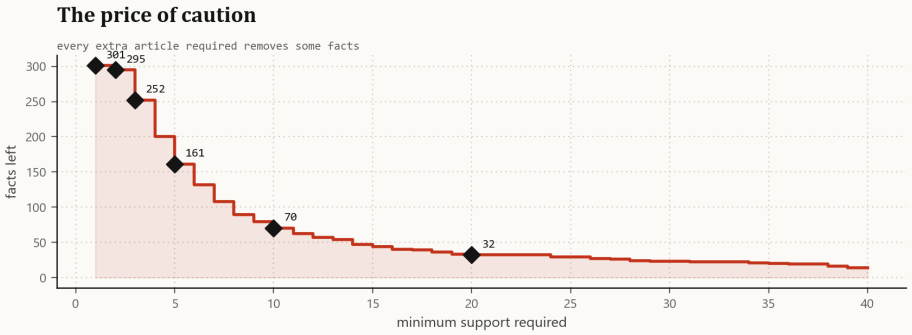


Figure 20. The price of caution: facts left at each evidence level.

Requiring three articles keeps 84% of the extracted facts and requiring ten keeps 23%, the core Vinmec repeats everywhere. (The six facts at support 1 come from disease pages.) Because support is stored on every fact, nobody has to fix this choice for everyone: a query or a website visitor picks the level that suits the purpose.

PART 9

A Tour of Glyco Atlas

Screenshots of the running site, connected to the Neo4j database above and taken with headless Edge. Each scenario answers one question of Table 2.

9.1 Finding your way in

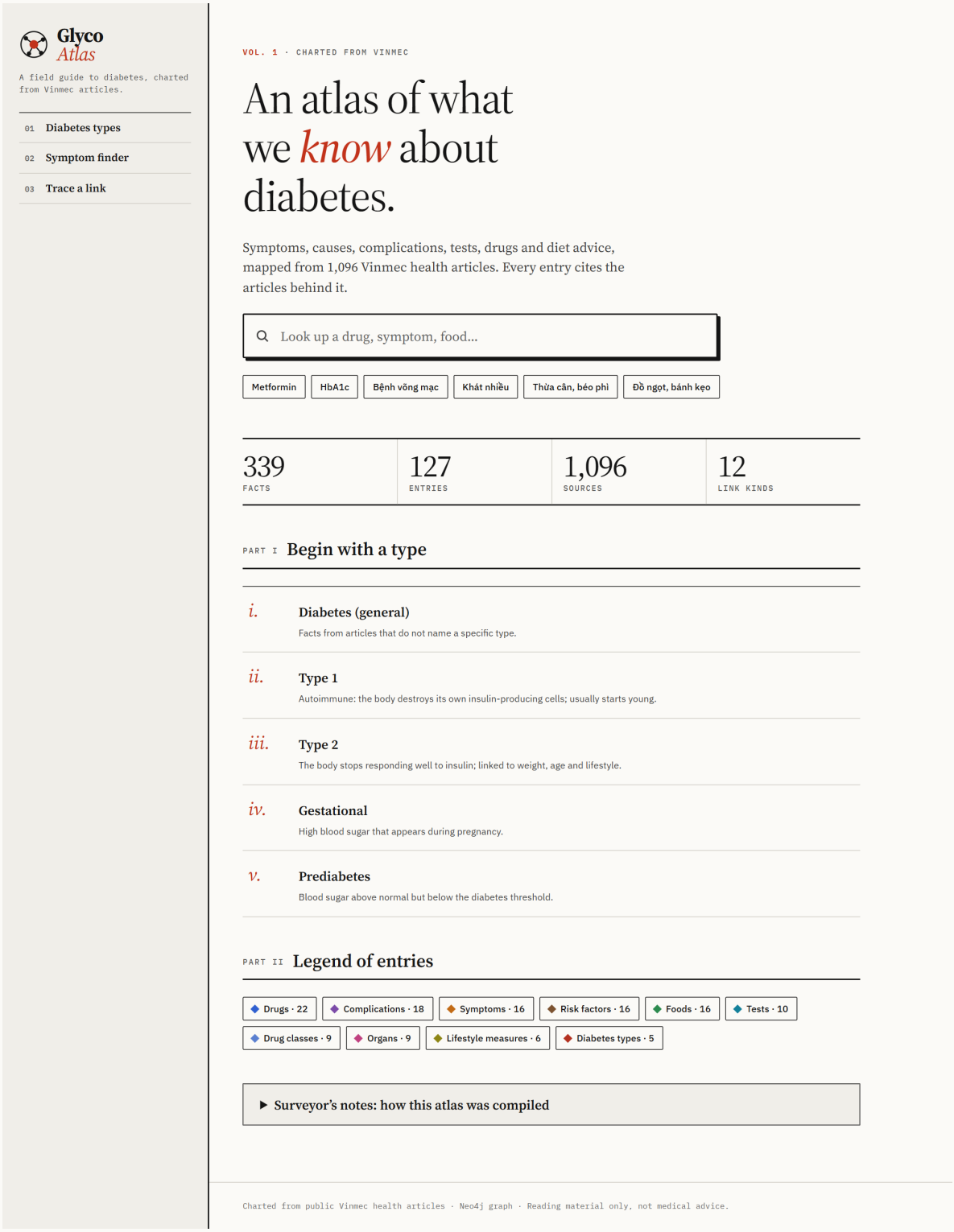


Figure 21. Home: a search box, quick links, the size of the graph, and a way in for each diabetes type.

The home page (Figure 21) says in one sentence what the atlas is and offers three doors: search, an example entity, or a diabetes type. Four tiles give the scale (339 facts, 127 entries, 1,096 sources, 12 kinds of link), and a collapsible note explains in four steps how the atlas was compiled.

Symptoms, causes, complications, tests, drugs and diet advice, mapped from 1,096 Vinmec health articles. Every entry cites the articles behind it.

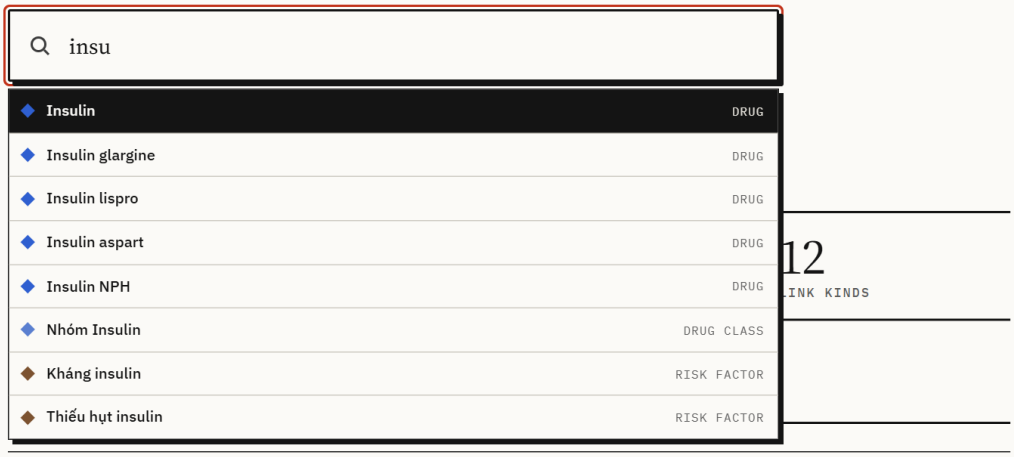


Figure 22. Typing "insu": suggestions appear as you type, each with its type.

Typing "insu" (Figure 22) lists every entity whose name contains it: five insulin drugs, the insulin class and two risk factors. A coloured diamond and a type label stop *Kháng insulin* being taken for a drug. Names that *start* with the text come first, then the best connected. Arrow keys and Enter work as in any search field, and each keystroke cancels the previous request so fast typing never shows stale results.

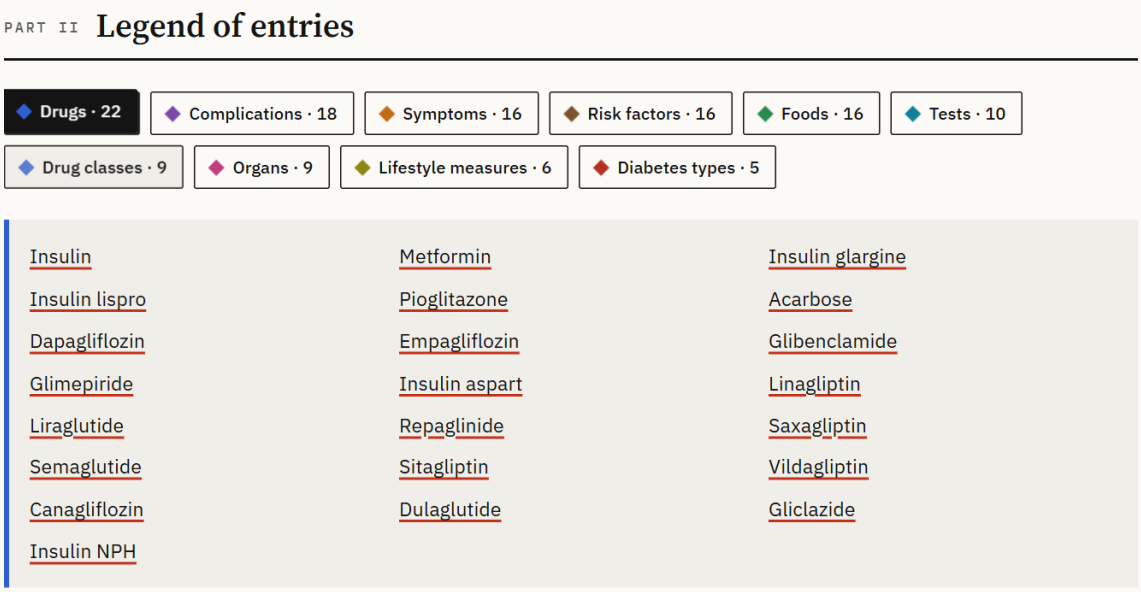


Figure 23. The legend of entries: each kind is a button that lists its entries.

The legend (Figure 23) is also a doorway: choosing a kind, here *Drugs*, lists every drug as a link to its page.

9.2 Everything about one entity, with its sources

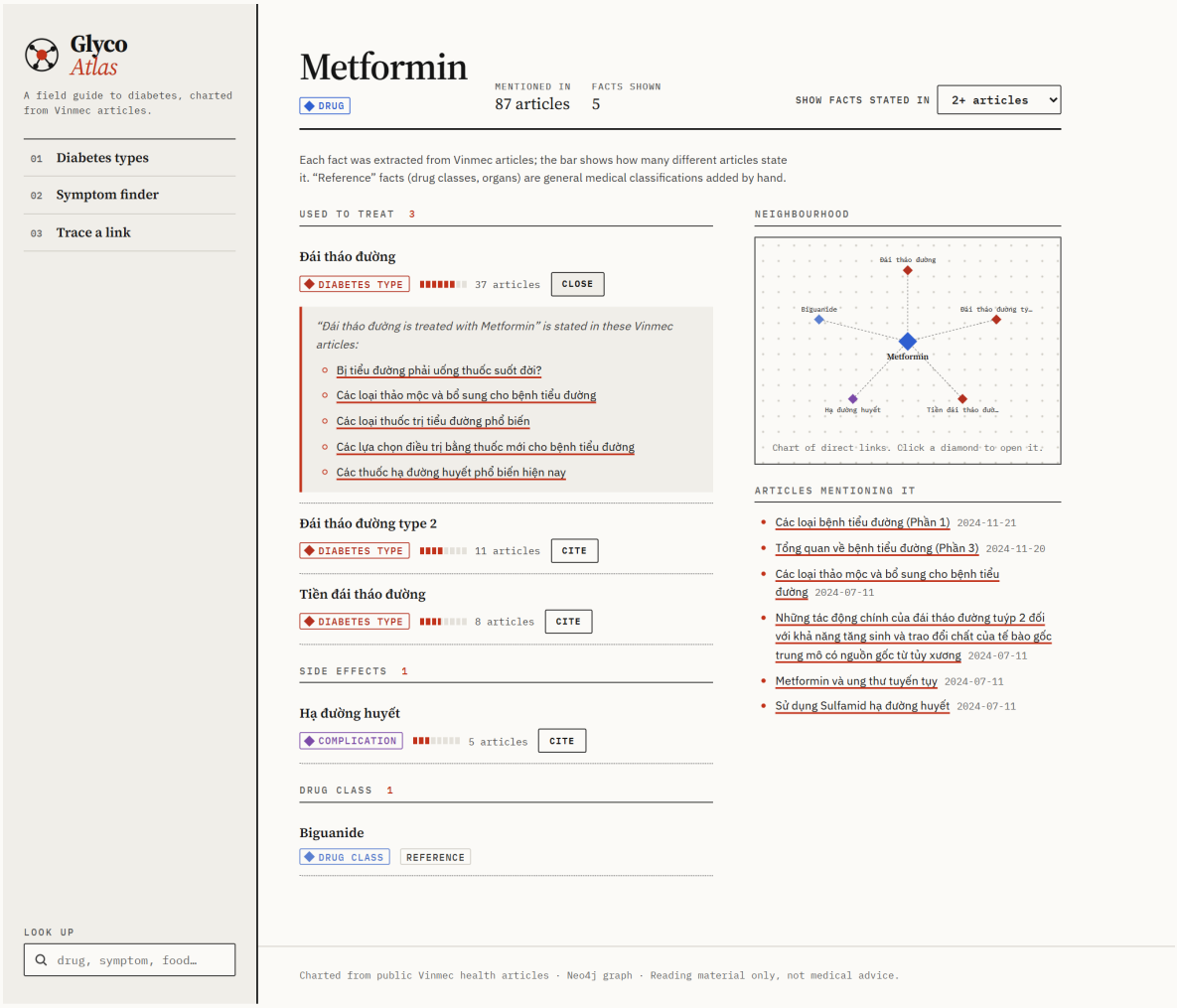


Figure 24. The Metformin page with the sources of the first fact open.

Choosing *Metformin* opens its page (Figure 24). The head shows its type, the 87 articles that mention it and the facts shown. Facts sit under headings derived from relation and direction (*Used to treat*, *Side effects*, *Drug class*). Each carries a segmented bar and a number for the articles stating it; the bar is logarithmic so 5 articles stays visible next to 37, but the number is always shown because the bar is decoration.

Cite opens the evidence for one fact: the sentence "Đái tháo đường is treated with Metformin" followed by links to five articles, the provenance query of Part 7 offered to any reader. The class carries a *reference* tag instead of a bar because it was added by hand. On the right, the neighbourhood (up to twelve strongest neighbours, diamonds coloured by type, each a link) lets people click from entity to entity, and the latest mentioning articles follow. The selector *Show facts stated in* hands over the precision/recall choice.

9.3 Comparing the types

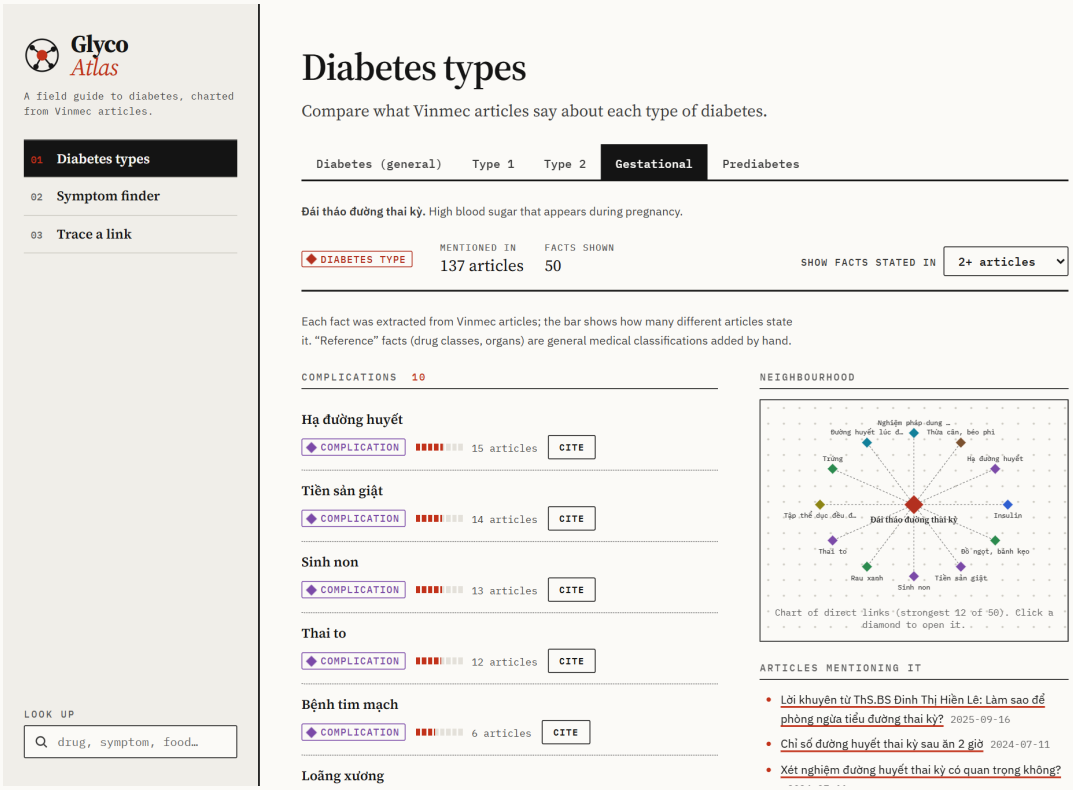


Figure 25. The Diabetes types page with the gestational tab open.

Five tabs, each with a plain one-line explanation (Figure 25). In the gestational tab the complications start with hypoglycaemia (15 articles), pre-eclampsia (14), premature birth (13) and a large baby (12), the pregnancy pattern of Figure 14.

9.4 Checking symptoms

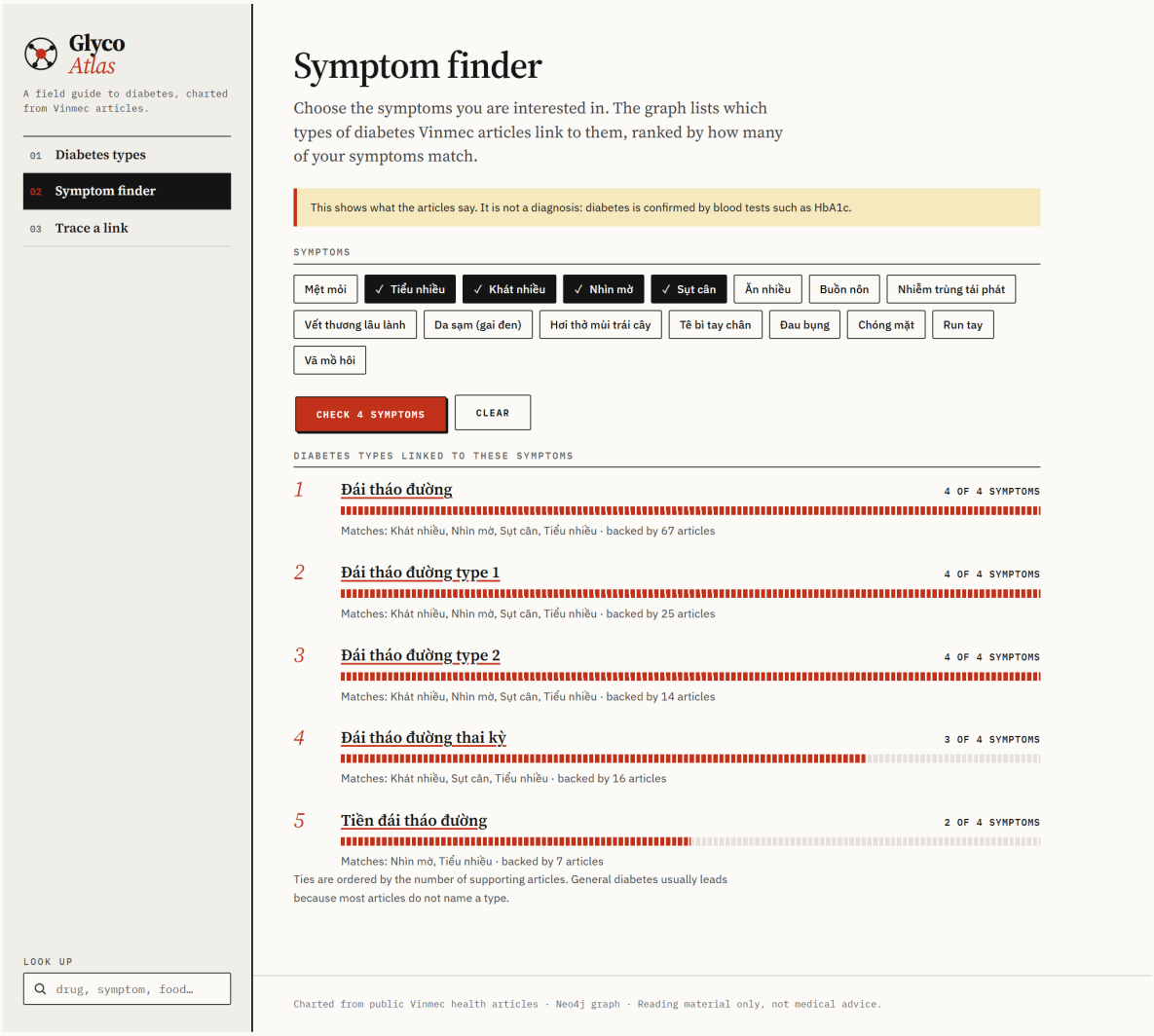


Figure 26. The symptom finder with four symptoms chosen and the ranked result.

On the symptom finder (Figure 26) the visitor picks from the 16 symptom entities and presses *Check*. Types are ranked by how many chosen symptoms the articles tie to them ("4 of 4"), with a segmented bar, the matched list and the supporting article count. A notice states this shows what the articles say and is not a diagnosis, because blood tests such as HbA1c confirm diabetes. It is the query of Table 18, readable without knowing the graph exists.

9.5 Tracing a link

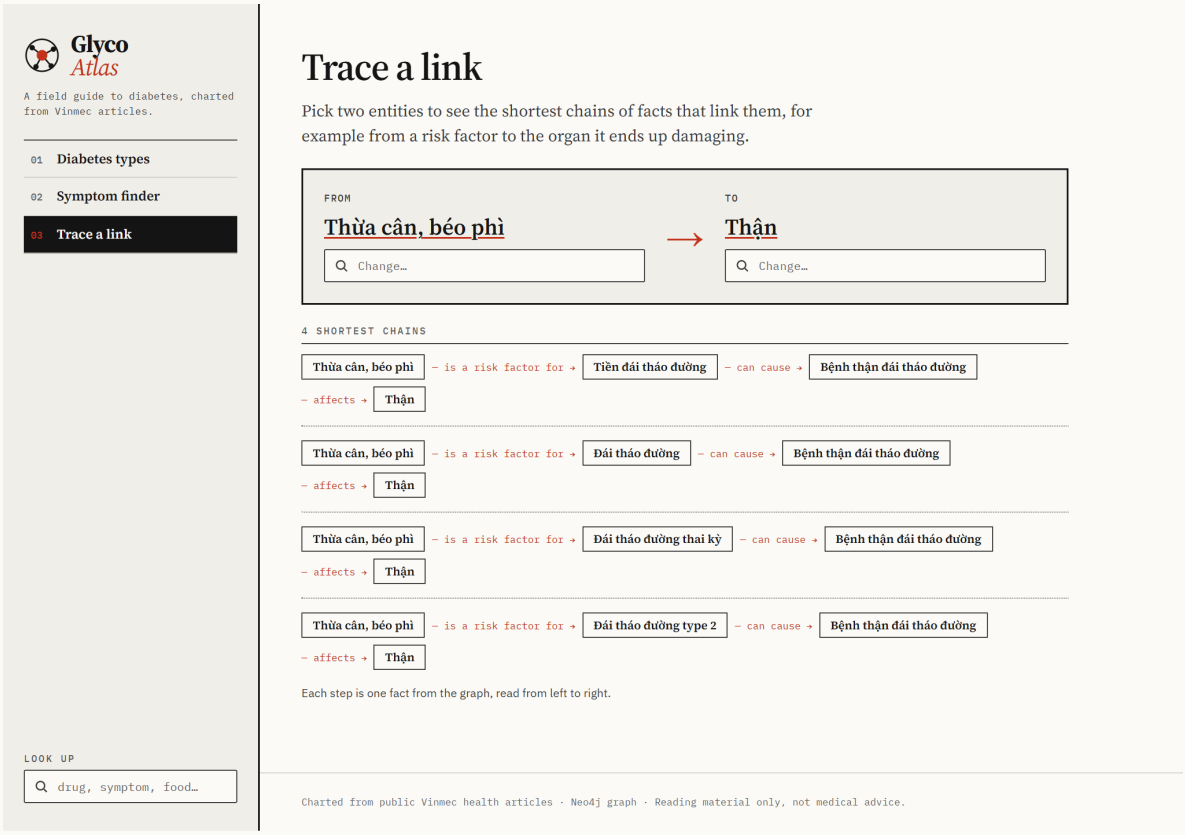


Figure 27. Trace a link: the shortest chains from obesity to the kidney.

Trace a link (Figure 27) takes two entities and returns the shortest fact chains between them. From obesity to the kidney there are four chains of three steps, one through each diabetes type that lists obesity as a risk factor, for example:

> *Thừa cân, béo phì* — is a risk factor for → *Đái tháo đường type 2* — can cause → *Bệnh thận đái tháo đường* — affects → *Thận*.

Each step is one fact phrased in the right direction: the stored fact is "type 2 `HAS_RISK_FACTOR` obesity", and since the path walks it backwards the site says "is a risk factor for". Each step comes from different articles, and the graph joins them because they share a node.

9.6 On a phone



Figure 28. The HbA1c page at phone width: one column, a sideways-scrolling menu.

Below 960 px the side rail folds into a header band (Figure 28): the entity page becomes one column, the search box moves under the wordmark and the chapters scroll sideways. The HbA1c page also shows its diagnostic threshold ($\geq$ 6.5%) in the head, the one place a node property is displayed.

PART 10

Where It Goes Wrong

The check of Part 8 found two wrong facts in fifty. This part says why, what changed, what the change cost and which other errors remain.

10.1 The two errors

Table 26. The two wrong facts and what the articles actually said.

WRONG FACT	WHAT THE SUPPORTING ARTICLES SAID
Đái tháo đường HAS_RISK_FACTOR Rối loạn mỡ máu (support 16)	High cholesterol is a risk factor of the *kidney* and *eye* complications, under headings like “Những ai dễ bị bệnh thận đái tháo đường?” and “Ai có nguy cơ mắc bệnh võng mạc đái tháo đường?”, and a cause of itching (“Một số nguyên nhân gây ngứa bàn tay và bàn chân”)
Đái tháo đường HAS_SYMPTOM Đau bụng (support 18)	Abdominal pain is a symptom of *ketoacidosis* and of nerve damage in the gut, under headings like “Biến chứng thần kinh” and “Kiểm soát biến chứng và phục hồi chức năng”

Same cause: a fact belonging to a **complication** was attached to the disease. The rule allowed it because the headings held an allowed keyword (*nguy cơ*, *nguyên nhân*) or, in version one, complication sections could also yield symptoms. It asked *what kind* of information a heading announces but not *whose* symptoms or risk factors they are.

Two observations made the diagnosis clear:

- Both facts had **high support** (16 and 18), so the support filter could not catch them: many articles discuss complications, and each cast a wrong vote. Agreement removes *random* noise, never an error the rules produce *systematically*.
- Both sat in the two relations where *ownership* matters. A complication always belongs to diabetes; a symptom may belong to diabetes or to one of its complications.

10.2 The fix

The `about_disease()` gate of Part 4.3.4: for symptoms and risk factors the heading must name diabetes or be short and generic, must not name another topic and must not be about reducing a risk; complication sections no longer yield symptoms. Re-extraction took the graph from 345 to 339 facts.

10.3 What the fix cost: precision against recall

Table 27. The six facts removed by the stricter rule.

REMOVED FACT	WAS THE REMOVAL RIGHT?
type 1 HAS_SYMPTOM Tê bì tay chân	yes: numbness is the nerve complication, not a symptom of type 1
type 2 HAS_SYMPTOM Vã mồ hôi	yes: sweating is a low-blood-sugar symptom
Tiền đái tháo đường HAS_RISK_FACTOR Tuổi cao	no: a true fact (sample #5) lost
Tiền đái tháo đường HAS_RISK_FACTOR Tăng huyết áp	no: a true fact (sample #15) lost
Đái tháo đường HAS_RISK_FACTOR Dùng corticoid kéo dài	no: a true fact lost
type 2 HAS_RISK_FACTOR Hút thuốc lá	no: a true fact lost

Two of the six removals were real errors the sample had not happened to contain. The other four were true facts. Each had one good supporting article and one weak one (the age and blood-pressure facts for prediabetes came partly from *"Chăm sóc các vấn đề tim mạch ở người bệnh tiểu đường và tiền tiểu đường"*, whose risk bullets sit under a heart-risk heading). The stricter rule dropped the weak vote, support fell to 1, and the two-article minimum removed the fact. One visible result (Table 6): the risk factor *Dùng corticoid kéo dài* lost its only fact and left the graph.

And the original two errors? They *stayed*, with far less support (16 → 3 for high cholesterol, 18 → 6 for abdominal pain). The remaining articles do state them under genuine diabetes headings ("Nguyên nhân và yếu tố nguy cơ gây bệnh tiểu đường: … Rối loạn mỡ máu"; abdominal pain in symptom lists for children, where it signals ketoacidosis at onset). By the definition used here, they are now properly supported; the weakness had been the weak sources, not the facts.

THE TRADE-OFF EVERY EXTRACTOR FACES

> Strict rules give fewer wrong facts but miss more true ones; loose rules find more true facts and let in more wrong ones. No setting suits every use, which is why support stays on every fact instead of serving once as a filter: a careful application can demand three or five articles, someone exploring can see everything, and the website hands that choice to its visitors.

10.4 Other known errors

A fifty-fact sample cannot find every error. Reading the queries and the graph revealed four more.

A wrong side effect. ♦The graph holds *Insulin HAS_SIDE_EFFECT Nhiễm toan ceton* (ketoacidosis), support 5. It is wrong: ketoacidosis comes from a *lack* of insulin and is *treated* with it. The *thiếu / không đủ* filter misses these phrasings, where "gây" belongs to something other than insulin:

- insulin as the **treatment**: "nhiễm toan ceton … có thể *gây* nguy hiểm đến tính mạng nếu như không được xử trí kịp thời bằng insulin";
- another drug causing it **together with** insulin: "Danazol: dùng cùng với Insulin do có thể *gây* tăng glucose máu và nhiễm toan ceton";
- hormones that **oppose** insulin: "Những hormone này … có tác dụng ngược lại với insulin … sẽ *gây* ra toan ceton".

Deciding *what* causes *what* needs the grammar of the sentence, which cue words cannot supply. Sentence-level extraction is the weakest part of the method (Part 12 proposes a language model).

An alias with a second meaning. ♦The fruit *táo* (apple) also matches *táo bón* (constipation). Word boundaries cannot help, as *táo* is a whole word in both. The section rule limits the damage (foods count only in diet sections), but a diet section that mentions constipation would cast a false vote for "low-sugar fruit".

A concept hidden by a longer one. ♦The vocabulary holds *Hạ đường huyết sơ sinh* (newborn hypoglycaemia) apart from *Hạ đường huyết*. Articles seldom use the long phrase; they say "trẻ sinh ra có thể bị hạ đường huyết", which matches the general entity. The newborn entity never reached two articles and is absent, and its facts count under the general one, which is why hypoglycaemia tops gestational diabetes' complications.

The general node dominates. ♦The subject comes from the title only, so the general node gathers most facts (Figure 7). No single fact is wrong, but type-specific questions are less complete than they could be.

PART 11

Known Blind Spots

The lecture names four ways a knowledge graph can fail: missing entities, wrong relationships, duplicates and outdated information. Each applies here in its own way, plus a few that come from design choices.

Missing entities (unknown recall). ♦The vocabulary is fixed at 106 concepts in 320 spellings. Whatever is not in it can never be found, however often the articles discuss it: a newer drug or insulin brand, a rarer complication, a folk remedy, a specific test. The evaluation measured precision, not recall; measuring recall means annotating articles by hand with every fact they state, which was out of scope.

Wrong relationships. ♦About 4% of sampled facts were wrong (likely range 1–13%), mostly a complication's fact attached to the disease. Headings are a strong signal of meaning, not a perfect one, and side effects read from single sentences are weaker still (Part 10).

Duplicates and granularity. ♦Aliases merge near-synonyms on purpose (*tê bì*, *dị cảm*, *kiến bò* become *Tê bì tay chân*) and sometimes merge what a doctor would keep apart, such as maternal and newborn hypoglycaemia. A spelling the vocabulary does not know is missed rather than duplicated, so true duplicates are rare.

Outdated information. ♦Every article carries a 2024 date that reflects Vinmec re-publishing its archive, so an old article weighs as much as a new one. Medical advice moves (new drug classes, revised targets) and the graph has no sense of time. A real system would re-crawl regularly and date each fact.

Coarse subjects. ♦The type comes from the title alone, so many type 2 facts sit on the general node.

Fixed food advice. ♦Whether a food is recommended or avoided is decided by the food, not read from the text, so "in moderation" and "choose low-sugar kinds" cannot be expressed.

One source. ♦Everything comes from one hospital's site. That keeps the graph consistent but inherits its emphases and gaps, and "correct" means "stated by Vinmec", not "checked against current clinical guidelines".

Hand-added facts. ♦Drug classes, affected organs and thresholds were added by hand. They are tagged `curated` and are standard medical knowledge, but they were not learned from the articles.

Not medical advice. ♦The graph summarises public health articles. The symptom finder shows which types the articles associate with symptoms; it diagnoses nobody, and the website says so where it matters.

PART 12

Takeaways and Next Steps

12.1 What was achieved

- **Data (R1).** 1,197 candidates found through the sitemap; a filter on the accented text removed accent collisions and off-topic pages, leaving 1,096 diabetes articles (1.13 million words), stored section by section.
- **Knowledge graph (R2, R3).** A vocabulary, the articles' own headings and agreement between articles produced 127 entities of 10 types and 339 facts of 12 relations, as `subject, relation, object` CSV with support and sources, well above the lecture's minimums.
- **Neo4j and queries (R4, R6).** Evidence on every relationship, articles as nodes, fifteen Cypher queries (one-hop, two-hop, aggregation, path, provenance) and a bounded `graph_facts` (5 facts at one hop, 230 at two).
- **Visualisation (R5).** Ring, radial, arc and ribbon views, per-type comparisons, degree and both PageRanks, article mentions.
- **Website (R7).** Glyco Atlas: search, facts with sources, type comparison, symptom finder and link tracing.
- **Evaluation (R8, R9).** 48 of 50 sampled facts correct (0.96; 0.87–0.99). The errors shared a cause, which was fixed; the fix's cost in lost true facts was measured and explained as a precision/recall trade-off.

THE MAIN LESSON

> **Structure already present in the text can do most of the work.** Health articles are organised under standard headings. Using them to decide what kind of fact a sentence may hold, and asking independent articles to agree, produced a dependable graph without a trained model. The quality check also showed agreement's limit: it removes random noise but not an error the rules make systematically, and only reading the errors revealed that.

12.2 Next steps

1. **Measure recall.** Annotate a sample of articles with every fact they state and count how many the extractor finds; that is the project's largest unknown.
2. **Grow the vocabulary semi-automatically.** Frequent phrases under symptom or treatment headings that match no entry are candidates for new entities, for a person to approve.
3. **Better relations.** Use a language model sentence by sentence ("does this say X causes Y, and is X taken or lacking?"), checked against the section rule. That would fix the side-effect errors and allow new relations such as *food prevents complication*.
4. **Knowledge graph plus RAG.** Connect the graph to the retrieval-augmented chat system of Assignment 08: the graph supplies facts and paths, article chunks supply detail, the answer cites both.
5. **Freshness.** Re-crawl on a schedule, date each fact, and prefer recent evidence when articles disagree.

PART A

Rebuilding Everything

Run from the `graph_01/` folder. The whole stack (Neo4j, API, website) also starts with `docker compose up --build`.

```
# environment
python -m venv .venv && source .venv/bin/activate
python -m pip install -r requirements.txt

# 1. crawl (about 20 minutes, resumable)  -> dataset/raw/pages/*.json, dataset/raw/index.csv
python kg/crawl.py

# 2. extract                                -> dataset/processed/{entities,triples,articles,mentions}.csv
python kg/extract.py

# 3. Neo4j and load
docker compose up -d neo4j
docker compose run --rm load            # or: python kg/load.py

# 4. retrieval, drawings, quality sample
python kg/retriever.py "Metformin" --hops 1
python kg/visualize.py
python kg/evaluate.py          # draws a new sample; overwrites evaluation/manual_audit.csv

# 5. notebook (kernel "Python (graph_01)")
jupyter nbconvert --to notebook --execute --inplace notebook/01_diabetes_kg.ipynb

# 6. website
docker compose up -d api web            # http://localhost:5173 , API docs at :8000/docs

# 7. report figures, tables and PDF
python report/extract_figures.py && python report/make_tables.py && python report/take_screenshots.py
cd report && xelatex Graph_01.tex && xelatex Graph_01.tex
```

The Cypher queries of Part 7 are in `kg/queries.cypher` and can be pasted into Neo4j Browser at `http://localhost:7474` (user `neo4j`, password `diabetes123`).